PCMS-KIN · 亲属关系计算引擎

亲属关系的代数与命名

公理化体系、规范形与中文称呼生成教程

把“谁是谁的谁”写成一条字母链，用关系代数把它化简成规范形，再用一套形式语法把规范形翻译成中文称呼。本书同时给出公理、定理、 算法、词法表与可复现的验证结果。

THE ALGEBRA AND GRAMMAR OF KINSHIP
An Axiomatic Introduction to Chinese Kinship Terms

整理 PCMS-Kin 项目
依据 `kinship_formula.py` · `naming.py` · `demo/kinship.js`
版本 2026 年 11 月 · 修订版

P C M E Y

前　言

市面上的“亲戚称呼计算器”大多是查表程序：预先枚举几百种关系，再逐条给出称谓。查表有两个绕不开的困难。其一是组合爆炸：关系可以任意复合，“母亲的姐姐的儿子的妻子的……”每多一层，可能的组合就翻几倍，表永远不够大。其二是无法推理：查表程序并不知道“父亲的妻子是母亲”，它只是恰好把这一条录进了表里；一旦遇到表外情况，它就沉默。

本书走另一条路：先把亲属关系本身公理化，再让称谓成为定理与算法的推论。亲属关系被看成二元关系，关系的复合就是乘法，“或者”就是加法，“反过来”就是求逆。在这个框架下，“我父亲的妻子”不过是两个字母的乘积 $P_m M_f$，而“它等于我母亲”是一条可以证明的等式。至于“这个关系在中文里叫什么”，则是另一层：一个从规范形到称呼词的**形式语法**。

> **本书的三层结构.**
>
> **语言层**：人、关系、生成元、链、复合、并、逆、零与幺。这一层只依赖关系代数的一般事实。
>
> **公理层**：十三条关于父母、配偶、同胞、长幼与存在性的局部事实。由此推出“每人恰有两个父母”“同胞必为全同胞”“长幼是严格全序”，以及十条化简等式。
>
> **命名层**：把化简后的规范形按配偶位置分派，使用直系、旁系、姻亲的词法表，以及“段前缀 + 重锚”算法，生成中文称呼。命名层还要满足两条硬约束：末位词的辈分等于目标的辈分，末位词的性别等于目标的性别。

本书不是两篇文章的简单拼接。公理体系回答“关系为什么可以化简”，命名手册回答“化简之后叫什么”；两者之间需要一座桥：**类型化**。无性别的公理只告诉我们 $PC = S + I$，但中文称呼必须区分父/母、子/女、兄/姐、弟/妹。于是我们在每个生成元上附加“这一步走到的人的性别”，把无性别的关系式细化为带类型的词。类型化既保留了公理的代数骨架，又让命名算法有足够的信息判断“这一支是否可能”“这个词该用父还是母”。

本书的写作目标是“精准”而不是“博雅”。所有定义、定理、算法都尽量写成可检验的形式；所有例子都给出链、公式、规范形和称呼；所有词法表都注明适用范围与失败条件。书中的程序片段来自实际实现，验证结果来自对一百一十余万条链的穷举与二十余万条长链的随机测试。第三部分会详细报告这些测试，以及多组真实 BUG 的解剖。

阅读指南

只想查称呼的读者：先读第 1 章的 1.1 – 1.3，学会把关系写成链；再读第 3 章的 3.1 – 3.3，学会化简；然后直接跳到第 5 章，按“配偶在末尾/开头/中间”查阅；附录 A – C 是速查表。

想理解数学结构的读者：按第 1 章 → 第 2 章 → 第 3 章的顺序读。第 2 章给出十三条公理与主要定理；第 3 章把定理变成可执行的改写系统。第 4、5 章是这套代数在自然语言上的“表示论”。

想实现或修改程序的读者：重点读第 3 章的算法、第 5 章的伪代码与不变量、第 6 章的接口与测试。代码中的每一个分支都能在书里找到对应的定义或定理；反过来，书里的每条规则也能在代码中找到实现。

约定 本书用“我”或“ego”表示关系链的起点。链从左到右书写，左边的词先走。例如 $P_m E_m C_f$ 读作“父的兄的女”。链末位的性别字母是目标人的性别。
本书的“辈分”一律指整数 $\mathrm{p}(w) = \#P - \#C$：正数表示长辈，负数表示晚辈，零表示平辈。日常口语中的“高一辈/低一辈”与这个定义一致。

符号表

表 0.1　常用符号

符号	含义	备注
$\mathcal{P}$	人的集合	关系的主体
P, C, M, E, Y	父母、子女、配偶、兄姐、弟妹	$C = P^{-1}$，$Y = E^{-1}$
S	兄弟姐妹	$S = E + Y$
$I, 0$	自己、空关系	幺元与零元
P_s, C_s, M_s, E_s, Y_s	带性别标注的生成元	$s \in \{m, f\}$，描述目标性别
$w = g_1 \cdots g_n$	链（词）	从 ego 走到目标
$\mathrm{p}(w)$	辈分	$\#P - \#C$
$\sigma(w)$	目标性别	末位生成元的性别
$\mathrm{seg}(X, g)$	段前缀	描述 X 与配偶 $M(g)$ 这一段
$\mathrm{rname}(g, s, Y)$	重锚名	把视角切到配偶后给 Y 的称呼
$\mu(T)$	称呼词的语义	返回（辈分，性别）

目　录

第一部分　关系代数与公理 …… 6

第 1 章　关系、链与类型 …… 7

1.1　亲属关系作为二元关系 …… 7
1.2　生成元与性别标注 …… 7
1.3　复合、并、逆、零与幺 …… 9
1.4　关系代数的基本定律 …… 9
1.5　类型化：良构链与零 …… 10
1.6　辈分、性别与阶 …… 10

第 2 章　十三条公理 …… 11

2.1　公理表 …… 12
2.2　父母与同胞 …… 13
2.3　长幼是严格全序 …… 13
2.4　十条化简等式 …… 14
2.5　化简表与八个不可约对 …… 15
2.6　模型、相容性与无穷性 …… 16

第 3 章　规范形与化简算法 …… 17

3.1　可接关系与十二条改写规则 …… 17
3.2　栈式化简算法 …… 18
3.3　六种规范形 …… 18
3.4　终止性、合流性与唯一性 …… 19
3.5　复杂度与实现要点 …… 19

第二部分　称呼的语法与算法 …… 21

第 4 章　称呼的语法与语义 …… 22

4.1　称呼作为词法项 …… 22
4.2　直系：P^n 与 C^n …… 22
4.3　旁系：$P^k\,E/Y\,C^j$ …… 23
4.4　姻亲：岳、公婆与眷姻 …… 24
4.5　段前缀与重锚 …… 25
4.6　同性婚姻的性别本位 …… 26
4.7　多解、主名与别名 …… 26

第 5 章　命名算法 …… 27

5.1　总调度：先看末尾 …… 27
5.2　配偶在末尾 …… 28
5.3　配偶在开头 …… 28
5.4　配偶在中间：切分与重锚 …… 29
5.5　兜底与绝对辈分 …… 30
5.6　正确性：辈分与性别不变量 …… 31
5.7　关系转换：A 如何称呼 B …… 32

第三部分　验证、实现与边界 …… 34

第 6 章　实现、验证与边界 …… 35

6.1　三个实现与公共接口 …… 35
6.2　穷举与随机验证 …… 35
6.3　典型 BUG 与系统性兜底 …… 36
6.4　边界与已知限制 …… 37
6.5　练习与核对清单 …… 37

附录 A　化简表与不可约对 …… 39

附录 B　眷姻段前缀表 …… 40
附录 C　重锚名速查 …… 40
附录 D　术语与符号 …… 41
附录 E　Lean 4 机器验证 …… 41

插图与表格

插图

1.1　五种基本关系与性别标注
1.2　辈分轴
3.1　改写系统的局部步骤
4.1　段前缀与重锚
5.1　命名流水线
5.2　关系转换

表格

0.1　常用符号
2.1　十条化简等式
3.1　相邻两步能否直接相连
3.2　十二条带类型改写规则
4.1　E/Y C^j 的基本词形
4.2　伯叔姑舅姨
4.3　堂表四线
4.4　常用段前缀
6.2　验证结果
A.1　无性别化简表
A.2　带类型改写表
B.1　常用段前缀基词
C.1　重锚名分类

PART I · 第一部分

关系为什么可以算

第一部分建立语言与公理。我们把亲属关系写成二元关系，把“的”写成乘法，把“或者”写成加法，把“反过来”写成求逆；然后用十三条局部事实推出父母、同胞、长幼与化简等式。读完之后，你会相信：一条长链可以被机械地化成一个短式，而这个过程不依赖任何一张称谓表。

第 1 章

关系、链与类型

Relations, Words, and Types

本章建立全书的语言。我们先把亲属关系写成二元关系，再把“沿关系走一步”写成生成元，把“连续走几步”写成链。随后引入性别标注，把无性别的关系式细化为带类型的词。读完本章，读者应当能把任意一句亲属关系翻译成一条链，并读出它的辈分与目标性别。

1.1 亲属关系作为二元关系

定义 1.1（亲属结构）. 一个亲属结构由非空集合 $\mathcal{P}$（人）与三个二元关系 $P, M, E \subseteq \mathcal{P} \times \mathcal{P}$ 组成：

$$p \xrightarrow{P} q \iff q \text{ 是 } p \text{ 的父母}, \qquad p \xrightarrow{M} q \iff q \text{ 是 } p \text{ 的配偶}, \qquad p \xrightarrow{E} q \iff q \text{ 是 } p \text{ 的兄姐}.$$

记号 $p \xrightarrow{R} q$ 读作“从 p 沿关系 R 走到 q”。又记 $\{R\}_p = \{q : p \xrightarrow{R} q\}$。

“子女”和“弟妹”不需要作为独立关系引入。它们是父母与兄姐的逆关系：

$$C := P^{-1}, \qquad Y := E^{-1}.$$

于是 $p \xrightarrow{C} q$ 当且仅当 $q \xrightarrow{P} p$，即 q 是 p 的子女。类似地，$p \xrightarrow{Y} q$ 当且仅当 $q \xrightarrow{E} p$，即 q 是 p 的弟妹。“兄弟姐妹”则是兄姐与弟妹的并：

$$S := E + Y.$$

这不是一条新公理，而是记号的展开：$p \xrightarrow{S} q$ 当且仅当 q 是 p 的兄姐或弟妹。

为什么要把 C, Y, S 定义成 P, E 的逆与并？因为“亲子互逆”“长幼互逆”是语言的语法，而不是关于世界的额外事实。把它们写成定义，公理就可以专心陈述真正有内容的东西：父母有几个、配偶是否唯一、同胞是否同父母、长幼是否可传递，等等。

定义 1.2（链）. 由生成元 P, C, M, E, Y 组成的有限序列

$$w = g_1 g_2 \cdots g_n, \qquad g_i \in \{P, C, M, E, Y\}$$

称为一条**链**。空序列记作 I（自己），并约定 I 是乘法的幺元。若某个关系不可能成立，记作 0（零）。链的**长度**是生成元的个数。

链的读法是“从左到右，每一步描述从当前位置再走一步”。例如

$$P_m E_m C_f$$

读作“父的兄的女”：先走到父，再走到父的兄，再走到兄的女。链的最后一个字母描述目标本人，因此目标的性别由末位字母决定。

1.2 生成元与性别标注

中文称呼必须区分父与母、子与女、兄与姐、弟与妹、夫与妻。因此在无性别生成元上附加一个性别字母 $s \in \{m, f\}$，得到**带类型生成元**：

$$P_s,\ C_s,\ M_s,\ E_s,\ Y_s \qquad (s \in \{m, f\}).$$

字母 s 永远描述“这一步走到的人”的性别，而不是“我”的性别。例如 P_m 是父，P_f 是母；E_m 是兄，E_f 是姐。

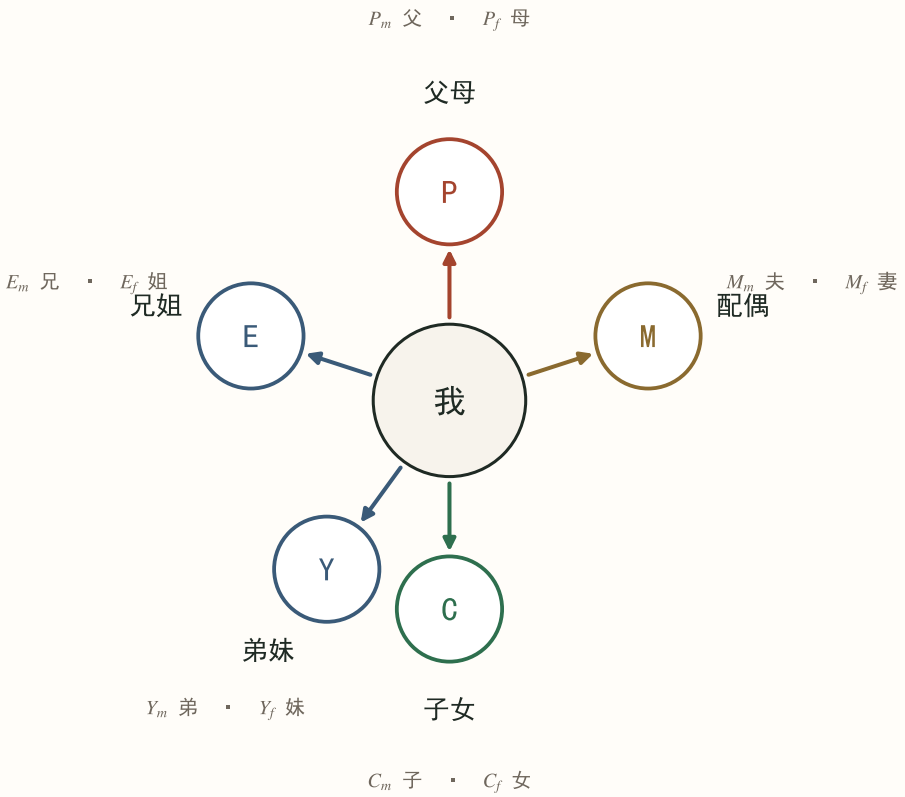


图 1.1 五种基本关系及性别标注。红、绿、蓝、棕分别表示长辈、晚辈、同辈与配偶；箭头从“我”指向目标。

定义 1.3（带类型链）. 由带类型生成元组成的有限序列

$$w = g_1 g_2 \cdots g_n, \qquad g_i \in \{P_m, P_f, C_m, C_f, M_m, M_f, E_m, E_f, Y_m, Y_f\}$$

称为**带类型链**。约定空链为 I。若链中某一步的性别与前后文矛盾，整条链解释为 0。

带类型链与无性别链的关系是“细化”：把每个无性别字母替换成带性别的一个实例，就得到带类型链。反过来，抹去所有性别字母，就得到无性别链。本书后面若不特别说明，写到 P, C, M, E, Y 时指无性别版本；写到 P_s 等形式时指带类型版本。

例 1.1. 下面几条链的读法：

中文	链	目标性别
爸爸	P_m	男
妈妈	P_f	女
爸爸的哥哥的儿子	$P_m E_m C_m$	男
妈妈的哥哥的女儿	$P_f E_m C_f$	女
妻子的妈妈	$M_f P_f$	女
哥哥的丈夫	$E_m M_m$	男

1.3 复合、并、逆、零与幺

定义 1.4（相等）. 两个关系 R_1, R_2 相等，记作 $R_1 = R_2$，当且仅当

$$\forall p,\quad \{R_1\}_p = \{R_2\}_p.$$

即：关系式由它所指的关系决定，与写法无关。PC、$S+I$、SS 可以是同一个关系的三种写法；第 2 章将证明它们相等。

定义 1.5（并）. $\{R_1 + R_2\}_p := \{R_1\}_p \cup \{R_2\}_p$。加法读作“或者”。

定义 1.6（复合）.

$$p \xrightarrow{R_1 R_2} r \iff \exists q,\ p \xrightarrow{R_1} q \ \wedge\ q \xrightarrow{R_2} r.$$

乘法读作“的”：先走 R_1，再走 R_2。

定义 1.7（逆）. $p \xrightarrow{R^{-1}} r \iff r \xrightarrow{R} p$。

定义 1.8（零与幺）. $\{0\}_p := \varnothing$，$\{I\}_p := \{p\}$。零是“此关系不成立”，幺是“自己”。

定义 1.9（幂）. $R^0 := I$，$R^{n+1} := R^n R$，$R^{-n} := (R^{-1})^n$。于是 P^2 是祖父母，P^3 是曾祖父母，$C^2 = P^{-2}$ 是孙辈，M^2 是“配偶的配偶”。

1.4 关系代数的基本定律

下面这些定律只依赖定义 1.4 – 1.8，不涉及任何亲属语义。它们是这套语言的语法，也是后面一切化简的形式基础。

引理 1.1（加法的交换、结合与零元）.

$$R_1 + R_2 = R_2 + R_1,\qquad (R_1 + R_2) + R_3 = R_1 + (R_2 + R_3),\qquad R + 0 = R.$$

证明. 逐点用并集的交换律、结合律，以及 $\varnothing$ 是并集单位元。□

引理 1.2（乘法结合律）. $(R_1 R_2) R_3 = R_1 (R_2 R_3)$。

证明. 对任意 p, r，

$$\begin{aligned} p \xrightarrow{(R_1R_2)R_3} r &\iff \exists q,\ p \xrightarrow{R_1R_2} q \ \wedge\ q \xrightarrow{R_3} r \\ &\iff \exists q,\ \big(\exists s,\ p \xrightarrow{R_1} s \ \wedge\ s \xrightarrow{R_2} q\big) \wedge q \xrightarrow{R_3} r \\ &\iff \exists s,\ p \xrightarrow{R_1} s \ \wedge\ \big(\exists q,\ s \xrightarrow{R_2} q \ \wedge\ q \xrightarrow{R_3} r\big) \\ &\iff p \xrightarrow{R_1(R_2R_3)} r. \end{aligned}$$

中段用的是 $\exists$ 的交换与 $\wedge$ 的结合律。□

由引理 1.2，乘法可以连写 $R_1R_2R_3$。这正是自然语言里“R_1 的 R_2 的 R_3”的对应。

引理 1.3（幺与零）. $IR = R = RI$，$0R = 0 = R0$。

证明. $p \xrightarrow{IR} r \iff \exists q,\ q = p \wedge q \xrightarrow{R} r \iff p \xrightarrow{R} r$；其余同法。□

引理 1.4（分配律）.

$$R_1(R_2+R_3)=R_1R_2+R_1R_3,\qquad (R_1+R_2)R_3=R_1R_3+R_2R_3.$$

证明. 把 $\exists q,(A_q\wedge(B_q\vee C_q))$ 拆成两个 $\exists$，反之合并。□

引理 1.5（逆）.

$$(R^{-1})^{-1}=R,\qquad (R_1R_2)^{-1}=R_2^{-1}R_1^{-1},\qquad (R_1+R_2)^{-1}=R_1^{-1}+R_2^{-1}.$$

证明. 以第二式为例：

$$p\xrightarrow{(R_1R_2)^{-1}}r\iff r\xrightarrow{R_1R_2}p\iff\exists q,\ r\xrightarrow{R_1}q\ \wedge\ q\xrightarrow{R_2}p\iff p\xrightarrow{R_2^{-1}R_1^{-1}}r.$$

其余同理。□

引理 1.5 的第二式说明：**求逆要反序**。“我的父亲的母亲”反过来是“我的孙子的儿子”，顺序整个倒过来。这条规则是第 5 章关系转换的基础。

1.5 类型化：良构链与零

无性别的关系式允许我们把 PC 写成 $S+I$，但中文称呼不能只说“兄弟姐妹”，还要说兄、弟、姐、妹。类型化的任务是给每个生成元附加目标性别，并规定哪些性别组合是可能的。

定义 1.10（良构链）. 带类型链 $w=g_1\cdots g_n$ 是**良构**的，如果存在一个人群模型与一条从 ego 到目标的路径，使每一步 g_i 的类型与性别都与路径上相应的人一致。否则 w 解释为 0。

良构性是语义概念；在算法上，我们用第 3 章的改写规则逐步检查它。直观上，良构性只关心两种冲突：

- **身份冲突**. I_s 要求当前人的性别为 s。若当前人已知为 $t\neq s$，则 $I_s=0$。
- **配偶冲突**. M_sM_t 要求“配偶的配偶”回到当前人，因此 t 必须等于当前人的性别；否则 0。

例 1.2. 设 ego 为男。

$E_mM_mM_f$：兄的丈夫的妻子。E_m 的目标是男；M_m 是这位兄的丈夫，仍是男；再 M_f 要求这位丈夫的妻子是女，但丈夫的配偶应是 ego（男），冲突，故整条链为 0。

$E_mM_mM_m$：兄的丈夫的丈夫。M_mM_m 回到 ego（男），故结果为 ego，即“自己”。

这段例子说明：类型化不是“给每个词加一个装饰”，而是把无性别代数中的 $M^2=I$ 细化为“$M_sM_t=I_t$，且仅当前一人性别为 t”。第 3 章会把这类规则整理成改写表。

1.6 辈分、性别与阶

定义 1.11（辈分）. 对链 $w=g_1\cdots g_n$，定义

$$\mathrm{p}(P_s)=+1,\quad \mathrm{p}(C_s)=-1,\quad \mathrm{p}(M_s)=\mathrm{p}(E_s)=\mathrm{p}(Y_s)=0,$$

并令 $\mathrm{p}(w)=\sum_i\mathrm{p}(g_i)$。$\mathrm{p}(w)>0$ 称长辈，<0 称晚辈，$=0$ 称平辈。

定义 1.12（目标性别）. 对非空链 $w=g_1\cdots g_n$，令 $\sigma(w)=s_n$，即末位生成元的性别。空链的性别记为“未定”。

辈分轴　$\mathrm{p}(P)=+1$，$\mathrm{p}(C)=-1$，其余生成元为 0

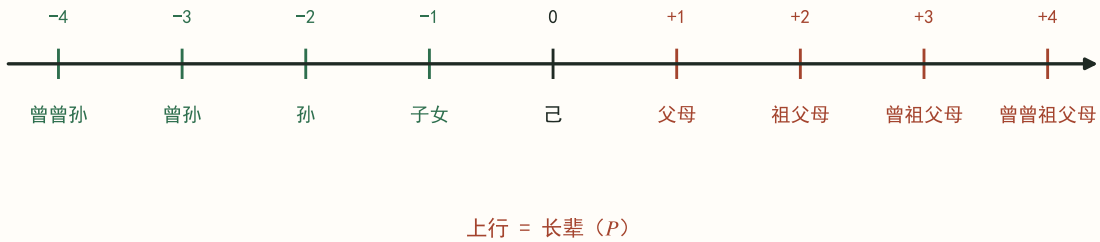


图 1.2　辈分轴。每走一步 P 加一，每走一步 C 减一；M, E, Y 不改变辈分。

引理 1.6（辈分可加）. $\mathrm{p}(uv)=\mathrm{p}(u)+\mathrm{p}(v)$。

证明. 按定义逐项相加。□

定义 1.13（阶与寸数）. 链的**阶**是生成元个数。直系血亲 P^n 的**寸数**为 $|n|$；旁系血亲 P^mSC^n 的寸数为 $m+2+n$。

阶是写法的属性，不是关系的属性：由 $S+I=SS$，同一个关系可以有阶 1 与阶 2 两种写法。寸数则是民间“几寸亲”的形式化；它与辈分同奇偶（第 2 章定理 2.16）。

小结 本章给出了三层对象：无性别关系式、带类型链、以及两个数值特征 p 与 σ。下一章将用十三条公理约束无性别关系式，得到可化简的代数；第 3 章再把化简规则细化到带类型链。

第 2 章

十三条公理

Thirteen Axioms for Kinship

本章列出十三条公理。它们地位平等，每条只陈述一个关于亲属世界的、可以直接点头的局部事实。第三节到第五节从这些公理推出三类结论：每人恰有两个父母、同胞必为全同胞、长幼在每个同胞类上构成严格全序；以及十条把长链化简为短式的等式。最后我们给出一个具体模型，说明十三条公理彼此相容。

2.1 公理表

公理 A1（双亲性）. 每人至少有两个互不相同的父母：

$$\forall p,\ \exists a\, b,\ a \neq b \ \wedge\ p \xrightarrow{P} a \ \wedge\ p \xrightarrow{P} b.$$

公理 A2（配偶对称）. $p \xrightarrow{M} q \implies q \xrightarrow{M} p$。

公理 A3（配偶唯一）. $p \xrightarrow{M} q \implies p \xrightarrow{M} r \implies q = r$。

公理 A4（无自婚）. $\neg\, p \xrightarrow{M} p$。

A2 – A4 合起来说：M 是 $\mathcal{P}$ 上的一个**匹配**（matching）——把所有人两两配对，每人至多属于一对，且不会与自己配对。

公理 A5（共子即配）.

$$p \neq q \implies p \xrightarrow{C} r \implies q \xrightarrow{C} r \implies p \xrightarrow{M} q.$$

即：共同养育一个孩子的两人互为配偶。

公理 A6（配偶共育）.

$$p \xrightarrow{M} q \implies p \xrightarrow{C} r \implies q \xrightarrow{C} r.$$

即：配偶的子女也是自己的子女。

A5 是十三条中唯一一条把 P 与 M 联系起来的公理，份量极重。A5 是单向的：它说“共子 ⇒ 配偶”，没说“配偶 ⇒ 共子”；反方向由 A6 补上。两条合起来说明：配偶对与父母对是同一回事。

公理 A7（同胞即共父母）.

$$p \xrightarrow{S} q \iff \Big(p \neq q \ \wedge\ \exists r,\ p \xrightarrow{P} r \ \wedge\ q \xrightarrow{P} r\Big).$$

把 S 展开，左端就是 $p \xrightarrow{E} q \vee q \xrightarrow{E} p$。A7 同时说了三件事：有共同父母的人是同胞；同胞必有共同父母；没有人是自己的同胞（从而没有人是自己的兄姐）。

公理 A8（同胞足数）. 每人至少有两个同胞：

$$\forall p,\ \exists a\, b,\ a \neq b \ \wedge\ p \xrightarrow{S} a \ \wedge\ p \xrightarrow{S} b.$$

公理 A9（长幼传递）. $p \xrightarrow{E} q \implies q \xrightarrow{E} r \implies p \xrightarrow{E} r$。

公理 A10（人人有兄姐）. $\forall p,\ \exists q,\ p \xrightarrow{E} q$。

公理 A11（人人有弟妹）. $\forall p,\ \exists q,\ q \xrightarrow{E} p$。

公理 A12（人人有子女）. $\forall p,\ \exists q,\ p \xrightarrow{C} q$。

公理 A13（人人有配偶）. $\forall p,\ \exists q,\ p \xrightarrow{M} q$。

冗余 A2、A4、A13 可由其余公理推出，独立的核是十条（证明见 §2.6 的注）。我们仍并列十三条，是因为每条都对应一个直观事实，便于先建立图像。追求极简时删去 A2、A4、A13 即可。

2.2 父母与同胞

定理 2.1（父母至多两人）. 若 a, b, c 是 p 的三个两两不同的父母，则矛盾。

证明. 由 A5，取 $(p, q) := (a, b)$、$r := p$。由 $a \xrightarrow{C} p$、$b \xrightarrow{C} p$ 得 $a \xrightarrow{M} b$。同法（取 (a, c)）得 $a \xrightarrow{M} c$。由 A3，$b = c$，与 $b \neq c$ 矛盾。□

定理 2.2（父母恰有两人）. $|\{P\}_p| = 2$ 对一切 p 成立。

证明. A1 给出 ≥ 2，定理 2.1 给出 ≤ 2。□

推论 2.3. 若 $p \xrightarrow{P} a$、$p \xrightarrow{P} b$、$a \neq b$，则 $a \xrightarrow{M} b$。

证明. 由 $a \xrightarrow{C} p$、$b \xrightarrow{C} p$ 与 A5。□

定理 2.4（全同胞）. 若 $p \xrightarrow{S} q$，则 p 与 q 的父母完全相同：

$$\forall r,\quad p \xrightarrow{P} r \iff q \xrightarrow{P} r.$$

证明. 由 A7 的左到右，取 a 使 $p \xrightarrow{P} a$、$q \xrightarrow{P} a$。由 A1 与定理 2.2，取 p 的另一个父母 $a' \neq a$，取 q 的另一个父母 $b' \neq a$。由推论 2.3，$a \xrightarrow{M} a'$ 且 $a \xrightarrow{M} b'$；由 A3 得 $a' = b'$。

现证双向。设 $p \xrightarrow{P} r$。由定理 2.2 与推论 2.3，$r = a$ 或 $r = a'$；前者由 $q \xrightarrow{P} a$，后者由 $q \xrightarrow{P} b' = a'$ 得证。反之同理。□

定理 2.4 是全文的枢纽。它把“同胞”彻底还原为“父母相同”，于是后面一切涉及 S 的推理都可以换成关于父母集合的等式。没有它，$SS = S + I$ 这类等式无从下手。

2.3 长幼是严格全序

定理 2.5（长幼反自反）. $\neg p \xrightarrow{E} p$；从而 $\neg p \xrightarrow{S} p$。

证明. 若 $p \xrightarrow{E} p$，由 A7 得 $p \xrightarrow{S} p$，再由 A7 得 $p \neq p$，矛盾。□

定理 2.6（长幼反对称）. $p \xrightarrow{E} q \implies \neg\, q \xrightarrow{E} p$。

证明. 否则由 A9 得 $p \xrightarrow{E} p$，与定理 2.5 矛盾。□

定理 2.7（同胞类上的严格全序）. 固定 p，令 $\mathcal{C}_p := \{S\}_p$。则

$$q \prec q' \iff q \xrightarrow{E} q'$$

在 $\mathcal{C}_p$ 上是严格全序。

证明. 四条性质：反自反即定理 2.5；传递即 A9；三歧：设 $q \neq q'$ 均在 $\mathcal{C}_p$ 中，由 A7 的右到左（$q \neq q'$ 且二者与 p 同父母）得 $q \xrightarrow{S} q'$，即 $q \prec q'$ 或 $q' \prec q$；唯一性由定理 2.6。□

由 A10、A11，这个全序没有最大元也没有最小元。于是每个同胞类都序同构于 $(\mathbb{Z}, <)$ 的某个区间；具体地说，是整条 $\mathbb{Z}$。

2.4 十条化简等式

本节只用到 A1、A3、A5、A7、A8。它们是“亲属计算器”的核心：把一个长式子换成更短、更接近日常说法的式子。

定理 2.8（父母的子女）. $PC = S + I$。

证明. ($\subseteq$) 设 $r \in \{PC\}_p$，即存在 q：q 是 p 的父母，r 是 q 的子女。后者即 q 是 r 的父母，故 q 是 p, r 的共同父母。若 $r \neq p$，由 A7 得 $r \in \{S\}_p$；若 $r = p$，则 $r \in \{I\}_p$。

($\supseteq$) 设 $r \in \{S + I\}_p$。若 $r = p$，由 A1 取 p 的父母 q，则 q 是 p 的父母且 p 是 q 的子女，故 $p \in \{PC\}_p$。若 $r \in \{S\}_p$，由 A7 取共同父母 q，则 $r \in \{PC\}_p$。□

定理 2.9（同胞的同胞）. $SS = S + I$。

证明. ($\subseteq$) 设 $r \in \{SS\}_p$，即存在 $q \in \{S\}_p$ 且 $r \in \{S\}_q$。若 $r = p$，则 $r \in \{I\}_p$。设 $r \neq p$。由定理 2.4，$\{P\}_p = \{P\}_q = \{P\}_r$。取 $a \in \{P\}_p$，则 $a \in \{P\}_r$，即 p, r 有共同父母；由 A7（$p \neq r$）得 $r \in \{S\}_p$。

($\supseteq$) 若 $r = p$，由 A8 取同胞 q，由定理 2.7 的对称性得 $p \in \{S\}_q$，故 $p \in \{SS\}_p$。若 $r \in \{S\}_p$，由 A8 取两个不同同胞，其中至多一个等于 r，取另一个记 q。则 $q \in \{S\}_p$、$q \neq r$。由定理 2.4，$\{P\}_q = \{P\}_p = \{P\}_r$，故 q, r 有共同父母；由 A7 得 $r \in \{S\}_q$。故 $r \in \{SS\}_p$。□

A8 不可省 若某个同胞类只有两个人 $\{p, r\}$，那么 r 是 p 的同胞，但“p 的同胞的同胞”要找到第三个同胞才成立。A8 要求每人有两个同胞，即每个同胞类至少三人。这就是“至少两个”而不是“至少一个”的原因。

定理 2.10（父母与同胞的配偶）. $PM = P$，$SP = P$。

证明（$PM = P$）. ($\subseteq$) 设 $r \in \{PM\}_p$，即 q 是 p 的父母、r 是 q 的配偶。由定理 2.2 取 $q' \neq q$ 也是 p 的父母；由推论 2.3 得 $q \xrightarrow{M} q'$，再与 $q \xrightarrow{M} r$ 用 A3 得 $r = q'$，故 r 是 p 的父母。

($\supseteq$) 设 r 是 p 的父母。由定理 2.2 取 $q \neq r$ 也是 p 的父母；由推论 2.3 得 $q \xrightarrow{M} r$，故 $r \in \{PM\}_p$。

（$SP = P$）由定理 2.4，$q \in \{S\}_p$ 时 $\{P\}_p = \{P\}_q$；反向由 A8 取同胞 q。□

定理 2.11（子女的同胞）. $CS = C$。

证明. (⊆) 设 $r \in \{CS\}_p$，即 q 是 p 的子女、r 是 q 的同胞。由定理 2.4，$\{P\}_q = \{P\}_r$，故 p 是 r 的父母，即 r 是 p 的子女。

(⊇) 设 r 是 p 的子女。由 A8 取 r 的同胞 q；由定理 2.4，$\{P\}_q = \{P\}_r \ni p$，故 q 是 p 的子女。故 $r \in \{CS\}_p$。□

定理 2.12（子女的父母）. $CP = M + I$。

证明. (⊆) 设 $r \in \{CP\}_p$，即 q 是 p 的子女、r 是 q 的父母。由 q 是 p 的子女得 p 也是 q 的父母。故 p, r 有共同子女 q。若 $r \neq p$，由 A5 得 r 是 p 的配偶；若 $r = p$ 则 $r \in \{I\}_p$。

(⊇) 若 r 是 p 的配偶：由 A12 取 p 的子女 q；由 A6，r 也是 q 的父母。故 $r \in \{CP\}_p$。若 $r = p$：由 A12 取 p 的子女 q，则 p, r 都是 q 的父母，故 $p \in \{CP\}_p$。□

定理 2.13（配偶的子女与配偶）. $MC = C$，$MM = I$，$M^3 = M$。

证明. 先证 $MM = I$。(⊆) 设 $q \in \{MM\}_p$，即 p 的配偶 s 以 q 为配偶。由 A2，q 也是 s 的配偶；由 A3（同以 s 为第一元）得 $q = p$。(⊇) 由 A13 取 p 的配偶 s，由 A2 得 s 以 p 为配偶，故 $p \in \{MM\}_p$。

再证 $MC = C$。(⊆) 设 r 是 p 的配偶 q 的子女。由 A6，r 是 p 的子女。(⊇) 设 r 是 p 的子女；由 A13 取配偶 q，由 A6 得 r 是 q 的子女。故 $r \in \{MC\}_p$。

最后 $M^3 = M(MM) = (MM)M = IM = M$。□

定理 2.14（兄姐的弟妹）. $EY = YE = S + I$。

证明（$EY \subseteq S + I$）. 设 $r \in \{EY\}_p$，即存在 q：q 是 p 的兄姐，r 是 q 的弟妹。后者即 r 以 q 为兄姐。于是 p 与 r 有共同的兄姐 q。由 q 是 p 的兄姐得 q 是 p 的同胞，由定理 2.4 得 $\{P\}_p = \{P\}_q$；同理 $\{P\}_r = \{P\}_q$。故 $\{P\}_p = \{P\}_r$，即 p, r 有共同父母。若 $r \neq p$，由 A7 得 $r \in \{S\}_p$；若 $r = p$ 则 $r \in \{I\}_p$。

(⊇) 设 $r \in \{S + I\}_p$。若 $r = p$，由 A10 取 p 的兄姐 q 即可。设 p 与 r 是同胞（故 $p \neq r$）。由 A10 取 p 的兄姐 q_0。由定理 2.4，$\{P\}_{q_0} = \{P\}_p = \{P\}_r$，故 q_0 与 r 是同胞。由 A7，q_0 是 r 的兄姐，或 r 是 q_0 的兄姐。分三种情形：若 $q_0 = r$，由 A10 取 q_0 的兄姐 q_1，由 A9 得 q_1 是 p 与 r 的兄姐；若 q_0 是 r 的兄姐，直接取 q_0；若 r 是 q_0 的兄姐，由 A9，r 是 p 的兄姐，取 r 的兄姐 q_1，由 A9 得 q_1 同时是 p 与 r 的兄姐。三种情形都找到同时是 p, r 兄姐的人，故 $r \in \{EY\}_p$。YE 同法。□

2.5 化简表与八个不可约对

表 2.1　十条化简等式（无性别版本）

原式	可化简为	依据	读法
PC	$S + I$	定理 2.8	父母的子女＝自己或同胞
SS	$S + I$	定理 2.9	同胞的同胞＝自己或同胞
PM	P	定理 2.10	父母的配偶＝父母
SP	P	定理 2.10	同胞的父母＝父母
CS	C	定理 2.11	子女的同胞＝子女
CP	$M + I$	定理 2.12	子女的父母＝自己或配偶
MC	C	定理 2.13	配偶的子女＝子女
MM	I	定理 2.13	配偶的配偶＝自己
EY	$S + I$	定理 2.14	兄姐的弟妹＝自己或同胞
YE	$S + I$	定理 2.14	弟妹的兄姐＝自己或同胞

把表 2.1 与幺元律合起来，就得到一个从左到右的改写系统：遇到表中左列的二元组，替换成右列；遇到 I 就消去；遇到 0 就整支归零。重复到不能改写为止，得到**规范形**。第 3 章会给出精确的算法。

定理 2.15（八个不可约对）. 在表 2.1 的改写系统下，不可再化简的二元组恰有八个：

$$PP,\quad PS,\quad MP,\quad MS,\quad SM,\quad CM,\quad SC,\quad CC.$$

证明. 逐一检查表 2.1 的左列，只有这八个二元组不在其中；再检查每个的任意非平凡分解都会引入表中左列（例如 PP 的 P 不能与 P 化简，PS 的 S 展开后会引入 E 或 Y，等等）。□

这八个二元组分别对应日常称谓里必须“专门有个词”的关系：祖父母、父母的兄弟姐妹、配偶的父母、配偶的兄弟姐妹、兄弟姐妹的配偶、子女的配偶、兄弟姐妹的子女、孙辈。这个观察是第 4 章词法表的出发点。

定理 2.16（寸数与辈分同奇偶）. 设旁系血亲 $R = P^m S C^n$（$m, n \geq 0$）。则

$$\mathrm{p}(R) = m - n, \qquad \mathrm{c}(R) = m + n + 2 = \mathrm{p}(R) + 2(n+1),$$

故 $\mathrm{c}(R) \equiv \mathrm{p}(R) \pmod 2$。

证明. 按辈分定义，P 每个 $+1$，C 每个 -1，S 为 0；寸数定义为 $m+n+2$。代入即得。□

2.6 模型、相容性与无穷性

定理 2.17（人口无限）. 满足全部十三条公理的人口 $\mathcal{P}$ 必为无限集。

证明（一）. 设某个同胞类 $\mathcal{C}$ 有限。由定理 2.7，$(\mathcal{C}, \prec)$ 是严格全序，而有限全序集必有最大元 m。由 A10，m 还有兄姐 q，即 $m \prec q$ 且 $q \in \mathcal{C}$，与 m 是最大元矛盾。所以每个同胞类都无限。

证明（二）. 由定理 2.2，每人恰有两个父母。由 A12，每人有子女，故由 A5 每人有配偶；再由 A2、A3、A4，配偶关系把 $\mathcal{P}$ 分成若干二元组。由 A5，任何人的两个父母是配偶；反之由 A6、A12，每个配偶对都有子女。由定理 2.4，这些子女构成一个完整的同胞类。于是“配偶对”与“同胞类”一一对应。设 $|\mathcal{P}| = n$ 有限，配偶对数为 t，则 $n = 2t$，同胞类数也为 t。由 A8，每个同胞类至少三人，故 $n \geq 3t = \tfrac{3}{2}n$，当 $n > 0$ 时不可能。□

这说明十三条公理描述的不是任何一个具体家族，而是一个理想的、双向无限的世系。作为“亲属计算器”的语义基础，这完全够用：计算器要算的是“关系如何复合”，而不是“世界上有多少人”。

例 2.1（一个模型）. 取 $\mathcal{P} = \mathbb{Z}$。把整数两两配对：

$$P_0 = \{0, 1\}, \qquad P_j = \{-j, j+1\}\ (j \geq 1).$$

这些二元组恰好把 $\mathbb{Z}$ 分完。定义

$$\kappa(0) = 0, \qquad \kappa(n) = \max\{k : 2^k \mid n\}\ (n \neq 0),$$

并令

$$\{P\}_n := P_{\kappa(n)}, \qquad x \xrightarrow{M} y \iff \exists j,\ \{x, y\} = P_j,$$

$$x \xrightarrow{E} y \iff \kappa(x) = \kappa(y) \wedge \mathrm{odd}(x) < \mathrm{odd}(y),$$

其中 $\mathrm{odd}(n) = n / 2^{\kappa(n)}$。这个模型满足全部十三条公理：每人恰有两个父母；$M$ 是一个匹配；同胞类恰是 κ 的一个纤维，故无限；长幼在每个纤维上是严格全序且无端点。于是十三条公理相容。□

从无性别到带性别 本章的公理与定理都是无性别版本。带类型的版本在第 3 章给出：把 P 细化为 P_m, P_f，把 S 展开为 E_m, E_f, Y_m, Y_f，并在 I 与 M^2 上增加性别一致性条件。无性别定理是带类型规则的“影子”：抹去性别后，带类型规则必须回到表 2.1。

第 3 章

规范形与化简算法

Normal Forms and Rewriting

第 2 章给出了无性别的化简等式。本章把它们细化成带类型的改写规则，并写成可执行的栈式算法。算法的输出是若干条规范形；每条规范形只含六种形状之一。后续的命名算法只对规范形工作，因此“先化简、后命名”不是实现细节，而是正确性的前提。

3.1 可接关系与十二条改写规则

带类型链的化简从左到右维护一个栈。新来一个生成元 B 时，先看它能否直接接在栈顶 A 后面。所谓“直接接”，是指这两步在现实中可以连着走，不需要先合并。

表 3.1　相邻两步能否直接相连

栈顶 A	新词 B	说明
空	任意	第一步总是可以压栈
P_s 或 M_s	P_t, E_t, Y_t	父的父、父的兄、配偶的母等，可以直接走
E_s, Y_s 或 C_s	C_t, M_t	兄的子、子的妻等，可以直接走
其他	—	必须改写

若 A 与 B 不能直接接，就弹出 A，按表 3.2 把二元组 AB 改写成一个或多个生成元，再对每个结果复制当前栈、依次压回，继续检查是否还能化简。表 3.2 是第 2 章化简等式的带类型版本；右列若有多个词，表示“每一种都可能”，不是“任选一个”。

表 3.2　十二条带类型改写规则

A	B	改写成	说明
P_s	C_t	$E_t,\ I_t,\ Y_t$	父的子＝兄、自己或弟
P_s	M_t	P_t	父的配偶＝父或母（按配偶性别）
M_s	C_t	C_t	配偶的子＝子或女
M_s	M_t	I_t	配偶的配偶＝自己；t 必须等于前一人的性别，否则 0
C_s	P_t	$M_t,\ I_t$	子的父/母＝配偶或自己
C_s	E_t	C_t	子的兄＝子
C_s	Y_t	C_t	子的弟＝子
E_s 或 Y_s	P_t	P_t	兄弟姊妹的父/母＝父/母
E_s	E_t	E_t	兄的兄＝兄
Y_s	Y_t	Y_t	弟的弟＝弟
E_s	Y_t	$E_t,\ I_t,\ Y_t$	兄的弟＝兄、自己或弟
Y_s	E_t	$E_t,\ I_t,\ Y_t$	弟的兄＝兄、自己或弟

此外还有三条幺元/零元规则：

$$I_s X = X, \qquad X I_s = \begin{cases} X, & \sigma(X) = s, \\ 0, & \sigma(X) \neq s, \end{cases} \qquad 0X = X0 = 0.$$

最后一条中 X 可以是任意生成元或链。I_s 的性别检查是类型化最容易被忽略的一步：它把无性别的 I 变成了一个“带性别的自己”，从而使“哥哥的丈夫的妻子”这类链有可能归零。

例 3.1（父的子）．$P_m C_m$：栈顶 P_m，新词 C_m，不能直接接。查表得 E_m, I_m, Y_m。三条规范形分别命名哥哥、自己、弟弟。若 $P_m C_f$，则得 E_f, I_f, Y_f，即姐姐、自己、妹妹。

例 3.2（哥哥的丈夫的妻子）. $E_mM_mM_f$。先压 E_m；M_m 可接在 E 后（兄弟的配偶），压栈；再读 M_f，M_mM_f 不能直接接。查表得 I_f；但栈中 E_m 的性别为 m，与 I_f 冲突，故整支为 0。这表示：哥哥的丈夫的妻子不可能是“我”，因为哥哥的丈夫的配偶应是哥哥或我（男），不可能是女性。

3.2 栈式化简算法

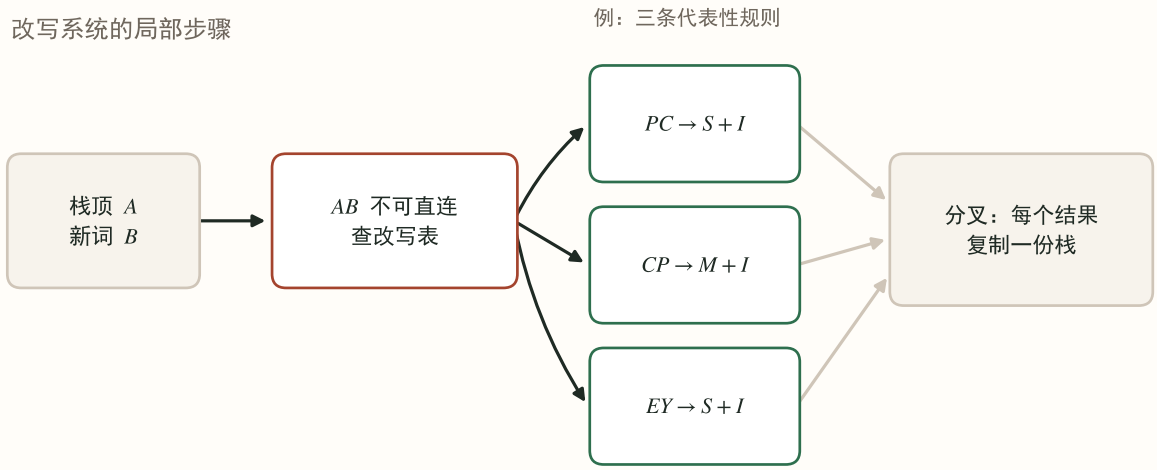


图 3.1 改写系统的局部步骤：弹出栈顶、查表、分叉、复制栈。每条分支独立继续化简。

算法 3.1（栈式化简）.

```
normalize(g[1..n]):
    branches ← { [] }                  # 每个元素是一条栈
    for i ← 1 to n:
        next ← ∅
        for each st in branches:
            next ← next ∪ push(st, g[i])
        branches ← dedupe(remove_zero(next))
    return branches

push(st, x):
    if st is empty or canPutIn(last(st), x):
        return { st + [x] }
    a ← pop_last(st)
    out ← ∅
    for each r in transform(a, x):     # 表 3.2 的右列
        out ← out ∪ push(st, r)        # 递归：r 可能又与前一项冲突
    return out
```

算法 3.1 的关键在 `push` 的递归：改写出来的词可能又和新的栈顶冲突，必须继续化简。例如 $P_mC_mP_m$ 先把 P_mC_m 改成 $E_m/I_m/Y_m$，再让每个结果与最后的 P_m 结合。手算时不要漏掉这一步。

`dedupe` 按词串完全相等去重，保留先出现的分支。`remove_zero` 删除含 0 的分支。最终得到的每个栈就是一条**规范形**。

3.3 六种规范形

化简后，每条非零规范形一定是下面六种形状之一。命名算法先判断形状，再进入对应章节。

表 3.3　六种规范形

形状	特征	例子
纯直系上	全部是 P	P_mP_f（奶奶）
纯直系下	全部是 C	C_mC_f（孙女）
纯旁系	不含 M，含 E 或 Y	$P_mE_mC_f$（堂姐妹）
配偶在末尾	末位是 M	E_mM_f（嫂子）
配偶在开头	首位是 M，末位不是 M	M_fP_m（岳父）
配偶在中间	首尾都不是 M，但含 M	$C_mM_fP_m$（亲家公）

若一条链同时以 M 开头和结尾，例如 $M_mE_mM_f$（丈夫的哥哥的妻子），它属于“配偶在末尾”还是“配偶在开头”？规则是：**先看末尾**。只要末位是 M，就按“配偶在末尾”处理；否则再看开头。这个优先级会影响后面第 5 章的分派顺序。

规范形还可以按“块”理解：上块 P^a、下块 C^b、旁块 P^aE/YC^b、配偶块 M。规范形的通式是

$$(\text{上块或旁块})\ M\ (\text{上块或旁块})\ M\ \cdots$$

每个 M 后面只能接 P 或 E/Y，不能接 C；每个 M 前面只能是 C 或 E/Y，不能是 P。这就是为什么规范形看起来总是“先上/旁，再配偶，再上/旁……”的节奏。

3.4　终止性、合流性与唯一性

定理 3.1（终止性）. 算法 3.1 在有限步内终止。

证明. 每次改写都把两个生成元换成一个生成元（或三条各含一个生成元的分支），因此每条分支的长度严格减少。递归 `push` 只对更短的栈调用，故终止。□

终止性只保证“能停”，不保证“停到同一个地方”。我们还需要**合流性**：若一条链可以走两条不同的改写路径，两条路径最终应得到同一组规范形。无性别版本的第 2 章等式已在 Lean 4 中机器验证（附录 E）；带类型版本在实现上通过以下三条工程约束保证：

- 改写只依赖栈顶与当前词的**局部**信息；
- 分支结果按固定顺序生成，最后按词串完全相等去重；
- 所有性别检查都是确定性的，不引入新的自由选择。

本书不给出带类型版本的完整合流性机器证明；第 6 章会报告穷举与随机测试的结果。对读者而言，可以把“规范形”理解为算法 3.1 的输出；只要同一输入在固定实现下输出稳定，命名层就可以工作。

定义 3.1（规范形）. 带类型链 w 的**规范形**是算法 3.1 在 w 上输出的、不含 0 的栈。若输出为空，则 $w = 0$。规范形的集合记作 $\mathrm{NF}(w)$。

3.5　复杂度与实现要点

设输入长度为 n，改写过程中最多产生 b 条分支。每条分支的每次 `push` 最多递归 n 层，因此算法的时间复杂度为 $O(bn^2)$，空间为 $O(bn)$。最坏情况下 b 可能随 n 指数增长（每次 PC 产生三条分支），但实际亲属链很短，且大量分支会在性别检查或去重时消失。

实现要点 （1）先复制栈再递归，避免分支之间互相污染。（2）改写结果要按表 3.2 的固定顺序生成，保证结果顺序稳定。（3）`I` 的性别检查必须在压栈前完成；若冲突，整支归零，而不是保留一个“待定”的 I。（4）去重按完整词串，不按名字；名字可能重复，词串不会。

例 3.3（完整化简）. $P_m C_m P_f$（爸爸的儿子的妈妈）：

```
P_m C_m     → E_m | I_m | Y_m
E_m P_f     → P_f
I_m P_f     → P_f
Y_m P_f     → P_f
```

三条分支都归到 P_f，去重后只剩 P_f，即妈妈。这个例子说明：分支可能很多，但最终规范形可能唯一。

PART II · 第二部分

从规范形到称呼

第二部分建立称呼的语法与算法。我们先把称呼看成带语义的词法项，再按直系、旁系、姻亲三类给出构词规则；随后把规则组织成“先分派、再段前缀、再重锚、最后兜底”的命名算法，并证明末位词的辈分与性别始终等于目标。

第 4 章

称呼的语法与语义

The Grammar of Kinship Terms

前两章把亲属关系化简成规范形。本章讨论“规范形如何变成中文称呼”。我们不再把称呼看成一张大表，而是看成一个带语义的词法系统：每个称呼词有辈分与性别，构词规则只允许在保持这两个量的前提下组合。这样，命名算法就可以像语法分析一样，从规范形的形状推出称呼。

4.1 称呼作为词法项

定义 4.1（称呼词法项）. 称呼词法项 T 由以下语法生成：

$$T ::= \text{父} \mid \text{母} \mid \text{子} \mid \text{女} \mid \text{兄} \mid \text{弟} \mid \text{姐} \mid \text{妹} \mid \text{夫} \mid \text{妻} \mid T\text{的}T \mid \text{眷}(T) \mid \text{姻}(T) \mid \cdots$$

实际词法表远比这条语法丰富；这里只列出后面用到的构造。称呼词法项不是任意字符串：它必须能通过词法表的组合规则生成。

定义 4.2（称呼的语义）. 对应称呼词法项 T，定义

$$\mu(T) = (\mathrm{gen}(T), \mathrm{sex}(T)) \in \mathbb{Z} \times \{m, f\},$$

其中 gen 是这个词所指对象的辈分，sex 是其性别。例如

$$\mu(\text{父}) = (+1, m), \quad \mu(\text{姐}) = (0, f), \quad \mu(\text{曾祖母}) = (+3, f).$$

命名算法的正确性要求：若规范形 w 被命名为 T，则

$$\mathrm{gen}(T) = \mathrm{p}(w), \qquad \mathrm{sex}(T) = \sigma(w).$$

这两条是本书反复使用的**辈分不变量**与**性别不变量**。第 5 章会把它们写成算法的断言，并在兜底路径上强制满足。

> **为什么不能简单相加** 中文称呼不是“每经过一个词就把辈分相加”的简单复合。例如“姨奶奶眷哥哥媳妇”里，“哥哥”是 0 辈，“媳妇”也是 0 辈，但整个词指的是 +2 辈的人。命名词法必须区分“段前缀”（描述锚点）与“末位词”（描述目标本身）。这就是 4.5 节“段前缀 + 重锚”的来源。

4.2 直系：P^n 与 C^n

4.2.1 直系长辈

设 $w = P_{s_1} P_{s_2} \cdots P_{s_n}$，性别序列 $s_1 s_2 \cdots s_n$ 从“我”这一侧向外排列。

- $n = 1$：P_m 是爸爸，P_f 是妈妈。
- $n = 2$：按 (s_1, s_2) 查表：

$$(mm) = \text{爷爷}, \quad (mf) = \text{奶奶}, \quad (fm) = \text{外公}, \quad (ff) = \text{外婆}.$$

- $n \geq 3$：逐代剥离。看最靠近“我”的一代 s_1：若为男，加“曾”；若为女，加“外曾”。剥掉这一代，继续。剥到只剩两代时，接“祖父/祖母/外祖父/外祖母”。

  例：

$$P_m P_m P_m = \text{曾祖父}, \qquad P_m P_f P_f = \text{曾外祖母}, \qquad P_f P_m P_m = \text{外曾祖父}.$$

这套规则不使用“高祖父、天祖父、烈祖父”等拟构词；四代及以上用“曾/外曾”长链直接写出，例如 P^4_{mmmm} 是“曾曾祖父”。

4.2.2 直系晚辈

与长辈完全对称。设 $w = C_{s_1} \cdots C_{s_n}$：

- $n = 1$：儿子/女儿。
- $n = 2$：孙子/孙女/外孙/外孙女，按 (s_1, s_2) 决定。
- $n \geq 3$：逐代剥离。最靠近“我”的一代若为男，加“曾”；若为女，加“外曾”。剥到剩两代，接“孙/孙女/外孙/外孙女”，末尾不再加“子”。

例：$C_m C_m C_m$ = 曾孙，$C_m C_f C_f$ = 曾外孙女。同样不使用“玄孙、来孙、云孙”等拟构词。

4.3 旁系：$P^k E/Y C^j$

旁系规范形由三段组成：先向上走 k 代（P^k），再走向某位祖先的兄弟姊妹（E 或 Y），再向下走 j 代（C^j）。形式写作

$$P^k\, E/Y\, C^j, \qquad k \geq 1,\ j \geq 0.$$

当 $k = 0$ 时，形状退化为 E/YC^j，即“我的兄弟姐妹及其后代”。

4.3.1 兄弟姐妹及其后代

表 4.1　E/YC^j 的基本词形

j	E/Y 为男	E/Y 为女
0	哥哥/弟弟	姐姐/妹妹
1	侄子/侄女	外甥/外甥女
2	侄孙/侄孙女	甥孙/甥孙女
≥ 3	侄 + 孙名	甥 + 孙名

4.3.2 父母的兄弟姐妹

$k = 1$ 时，$P_m E/Y$ 是父的兄/弟/姐/妹，$P_f E/Y$ 是母的兄/弟/姐/妹。词形表如下：

表 4.2　伯叔姑舅姨

链	称呼	链	称呼
$P_m E_m$	伯父	$P_m E_f$	大姑
$P_m Y_m$	叔叔	$P_m Y_f$	小姑
$P_f E_m$	大舅	$P_f E_f$	大姨
$P_f Y_m$	小舅	$P_f Y_f$	小姨

$k = 2$ 时，是祖辈的兄弟姐妹：伯公/叔公/姑奶奶/舅爷/姨奶奶等。$k \geq 3$ 时，用“线前缀 + 祖辈级”构造，例如“伯曾祖父”“舅曾祖父”。完整规则见附录 Λ 与《亲属称呼手册》第 5 章。

4.3.3 堂表亲与更深支系

$k = 1$ 且 $j \geq 1$ 时，目标是父母的兄弟姐妹的子女及其后代。关键是两条“线”：

表 4.3　堂表四线

$(P, E/Y)$	线	例
(m, m)	堂	$P_m E_m C_m$ = 堂兄弟
(m, f)	姑表	$P_m E_f C_f$ = 姑表姐妹
(f, m)	舅表	$P_f E_m C_m$ = 舅表兄弟
(f, f)	姨	$P_f E_f C_f$ = 姨姐妹

更深的支系用“从/族/再从”等前缀叠加。一般地，设 $r = j - k$ 为目标相对“我”的代数差：$r = 0$ 同辈，$r > 0$ 晚辈，$r < 0$ 长辈。命名时先由 $k = 1$ 或 $k = 2$ 的线表确定“线”，再由 r 确定“角色”（兄弟/姐妹、侄/侄女、伯叔/姑、侄孙……）。这个 r

就是第 2 章“寸数”与辈分差的来源。

> 例 4.1. $P_mE_mC_m$：父的兄的子。线为堂，$k=1, j=1, r=0$，角色为兄弟，故为“堂兄弟”（不区分堂哥/堂弟，因为链中没有年龄信息）。$P_mE_mC_mC_m$：$j=2, r=1$，角色为侄，故为“堂侄”。

4.4 姻亲：岳、公婆与眷姻

4.4.1 配偶在末尾

规范形末位是 M，即“某人的配偶”。按 X 的形状分派：

- $X = I$：丈夫/妻子。
- $X = E/Y$：嫂子/弟媳/姐夫/妹夫。
- $X = C^b$：儿媳/女婿/孙媳妇/孙女婿。
- $X = P^kE/Y$：伯母/婶婶/姑丈/舅妈/姨丈等。
- $X = P^kE/YC^j$：堂嫂/堂弟媳/堂姑丈等；深层用“配偶后缀表”。

4.4.2 配偶在开头

规范形首位是 M，且末位不是 M。最常见的是配偶的父母：

$$M_fP_m = \text{岳父},\quad M_fP_f = \text{岳母},\quad M_mP_m = \text{公公},\quad M_mP_f = \text{婆婆}.$$

配偶的兄弟姐妹：

$$M_fE_m = \text{大舅子},\quad M_fY_f = \text{小姨子},\quad M_mE_m = \text{大伯子},\quad M_mY_f = \text{小姑子}.$$

配偶的侄甥、堂表亲等，按“配偶视角”命名，再在必要时加“岳/公/婆”终端。

4.4.3 配偶在中间：眷与姻

规范形首尾都不是 M，但中间含 M，写作

$$X\,M(g)\,Y.$$

这里 X 是从“我”到某位亲属的链，$M(g)$ 是这位亲属的配偶，Y 是配偶的亲属链。命名分两步：

$$\mathrm{name}(XM(g)Y) = \mathrm{seg}(X, g) + \mathrm{rname}(g, s, Y),$$

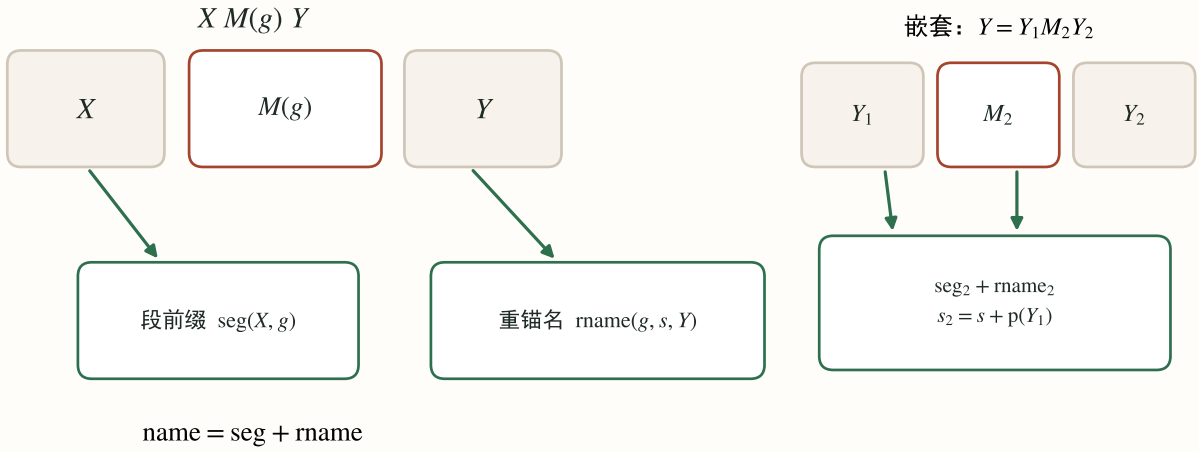
其中 $s = \mathrm{p}(X)$ 是 X 的辈分。seg 是**段前缀**，说明“这是 X 的配偶这一段的哪一类”；rname 是**重锚名**，说明“Y 是配偶的什么亲戚”。两者拼接，得到“X 的配偶的 Y”的完整名字。

表 4.4　常用段前缀（异性婚姻）

X 的形状	配偶女（$g=f$）	配偶男（$g=m$）
C_m	男眷	—
C_f	—	女姻
E/Y	兄弟眷	姊妹姻
C_mC_m	孙眷	—
C_mC_f	—	孙姻
E/Y_mC_m	侄眷	—
P_mE/Y_m	叔眷	—
P_fE/Y_m	舅眷	—
P_mE/Y_f	—	姑姻
P_fE/Y_f	—	姨姻

“眷”表示中间配偶是女性，“姻”表示中间配偶是男性。注意这是**段标记**，不是“眷属”的泛称。表中“—”表示旧版异性表未列出的组合；第 4.6 节会按“性别本位”补全同性组合。

4.5 段前缀与重锚



段前缀负责“这是哪一段”，重锚名负责“目标是什么”；嵌套时递归切分。

图 4.1 把 $XM(g)Y$ 拆成段前缀与重锚名。嵌套时，$Y = Y_1M_2Y_2$ 继续递归切分，新锚点辈分为 $s_2 = s + \mathrm{p}(Y_1)$。

定义 4.3（段前缀）. seg 输入 X 与配偶性别 g，输出一个段标记字符串。它只依赖 X 的形状、每个生成元的类别与性别，以及 g。段前缀先查有限表；未命中时用机械兜底：

$$\mathrm{seg}(X, g) = \mathrm{strip}(\mathrm{nameMateLast}(g, X)) + \text{眷/姻},$$

其中 strip 去掉配偶名的后缀。完整表见附录 B。

定义 4.4（重锚名）. 设配偶在 $s = \mathrm{p}(X)$ 代，Y 是配偶的亲属链。$\mathrm{rname}(g, s, Y)$ 是把视角切到配偶后，按 Y 的形状给目标起的名字。它的末位词必须满足

$$\mathrm{gen}(\mathrm{rname}) = \mathrm{p}(XM(g)Y), \qquad \mathrm{sex}(\mathrm{rname}) = \sigma(XM(g)Y).$$

嵌套调用时，若形状未被有限表覆盖，rname 至少返回目标的绝对辈分通用词，不能返回空。完整表见附录 C。

重锚名的计算按 Y 的形状分类：以 M 结尾；全是 P；单个 E/Y；P^kE/Y；E/YC^j；PE/YC^j；P^kE/YC^j（$k \geq 2$）；以及“Y 中还有配偶”的嵌套情形。每一类都有自己的辈分公式与词形。第 5.4 节会给出算法；附录 C 给出速查表。

例 4.2. $P_mE_mM_fC_m$（伯父的妻子的侄子）：

- 切第一个 M：$X = P_mE_m$（伯父），$g = f$，$Y = C_m$。
- $s = \mathrm{p}(X) = 1$。
- 段前缀：$X = P_mE_m$，配偶女 → “叔眷”。
- 重锚名：$Y = C_m$，$s = 1$，$j = 1$，$gen = s - j = 0$；目标男，同辈 → “侄”。
- 拼接：叔眷 + 侄 = **叔眷侄**。

4.6 同性婚姻的性别本位

这套系统不把“婚姻”限定为异性。生成元 M_m（夫）与 M_f（妻）可以接在任意性别的上一步之后；词法层按“性别本位”外推：**称呼由对方是男是女决定，不由“我”是男是女决定**。具体做法有三条：

1. **配偶并到父/母**：$P_s M_t = P_t$，即“父/母的配偶”按配偶性别直接变成父或母。同性双亲家庭中，“妈妈的妻子”仍是妈妈，“爸爸的丈夫”仍是爸爸。
2. **段前缀补全**：seg 的基词由 X 的性别决定（男/女、兄弟/姊妹、侄/甥、叔/舅/姑/姨、孙/外孙……），后缀由配偶性别决定（女 → 眷，男 → 姻）。例如 C_m+ 夫 → 男姻，C_f+ 妻 → 女眷，E/Y_m+ 夫 → 兄弟姻。
3. **配偶后缀**：同性配偶用“婿/媳/妇”等后缀。例如哥哥的丈夫＝兄婿，姐姐的妻子＝姐嫂，儿子的丈夫＝儿婿，女儿的妻子＝女媳。

> 例 4.3. $E_m M_m$：哥哥的丈夫。E_m 的目标是男，M_m 是男配偶，词法给“兄婿”。$E_m M_f$：哥哥的妻子，词法给“嫂子”。两者的区别只在配偶性别，不在“我”的性别。

4.7 多解、主名与别名

一条原始链可能化简成多条规范形，因此可能对应多个称呼。例如

$$P_m C_m \longrightarrow E_m \mid I_m \mid Y_m$$

分别命名为哥哥、自己、弟弟。计算器用“ / ”连接多个分支，并给出第一条为主名。多解不是程序错误，而是关系代数中“或者”的直接体现。

同一个称呼还可能有别名：爸爸/父亲/父，妈妈/母亲/母，哥哥/兄，姐姐/姊，等等。主名是最常用的那个；别名用于口语或正式场合。手册中的“主名”指计算器结果中斜杠分隔的第一项或唯一项。

> **年龄信息** E/Y 只表示“上一步那个人的哥哥/姐姐/弟弟/妹妹”，也就是兄弟姊妹之间的长幼。这套记号没有“堂哥/堂弟”这种同辈旁系的长幼信息，也没有“表哥/表弟”的信息。因此 $P_m E_m C_m$ 的主名是“堂兄弟”，不是“堂哥/堂弟”；别名里可以给出“堂哥、堂弟”，但主名不会变。

第 5 章

命名算法

The Naming Algorithm

本章把第 4 章的词法规则组织成一个可执行的算法。算法的输入是带类型规范形，输出是称呼；它先按配偶位置分派，再按形状调用直系、旁系或姻亲的词法，最后用“绝对辈分兜底”保证两条不变量。我们给出伪代码、正确性证明梗概，以及关系转换的算法。

5.1 总调度：先看末尾

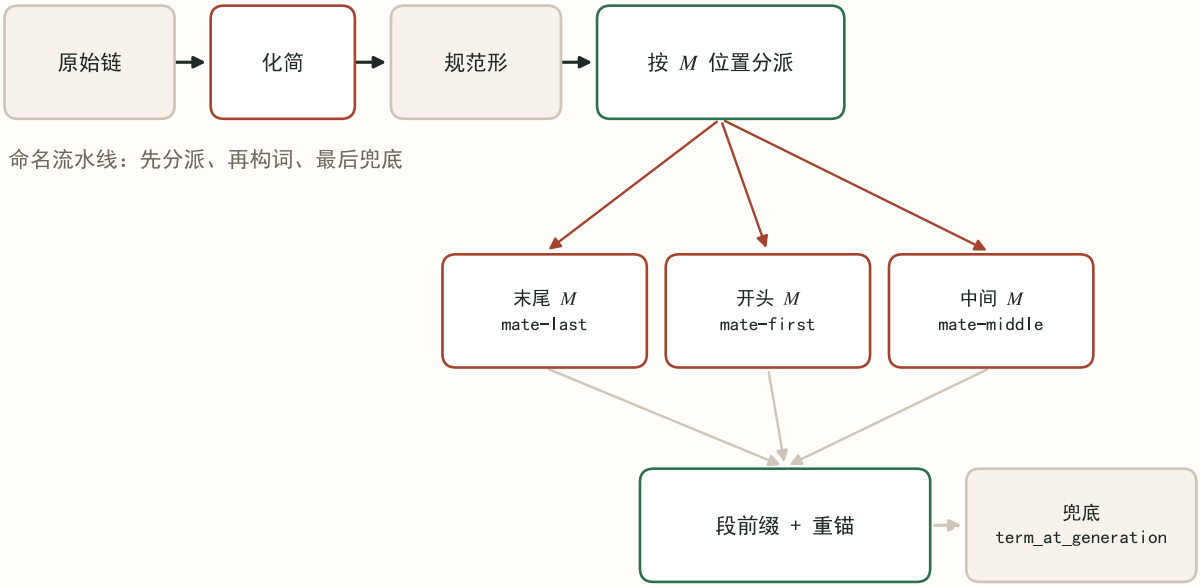


图 5.1 命名流水线：原始链先化简为规范形，再按 M 的位置分派到三个子算法；段前缀与重锚失败时走绝对辈分兜底。

算法 5.1（命名总调度）.

```
name(w):
    for each nf in normalize(w):
        yield nameNF(nf)

nameNF(w):
    if w = I:            return 自己
    if last(w) = M:      return mateLast(w)
    if first(w) = M:     return mateFirst(w)
    if M occurs in w:    return mateMiddle(w)
    if all P:            return ancestors(w)
    if all C:            return descendants(w)
    return collateral(w)          # E/Y 开头的纯旁系
```

分派顺序不能交换。若一条链同时以 M 开头和结尾，例如 $M_m E_m M_f$，必须先按“末尾 M”处理，否则会把“丈夫的哥哥的妻子”误当成“配偶的亲属”而得到不同结果。这就是第 3.3 节说的“末尾优先”。

引理 5.1（化简保持辈分与性别）. 若 $v \in \mathrm{NF}(w)$，则

$$\mathrm{p}(v) = \mathrm{p}(w), \qquad \sigma(v) = \sigma(w).$$

证明. 逐条检查表 3.2：每条改写规则左右两端的辈分相同（即第 2 章定理 2.8 – 2.14 的带类型版本），且末位生成元的性别就是目标的性别。□

5.2 配偶在末尾

设规范形为 $XM(g)$。目标是 X 的配偶，因此

$$\mathrm{p}(XM(g)) = \mathrm{p}(X), \qquad \sigma(XM(g)) = g.$$

命名按 X 的形状分派：

- $X = I$：丈夫/妻子。
- $X = E/Y$：兄弟的配偶（嫂子/弟媳/姐夫/妹夫）。
- $X = C^n$：子女/孙辈的配偶（儿媳/女婿/孙媳妇/孙女婿……）。
- $X = P^k E/Y$：伯叔姑舅姨的配偶（伯母/婶婶/姑丈/舅妈/姨丈……）。
- $X = P^k E/Y C^j$：堂表亲的配偶（堂嫂/堂弟媳/堂姑丈……）。
- X 内含 M：先按第 5.4 节的“段前缀 + 重锚”命名 X，再套配偶后缀；若段前缀不可用，则按 X 的末位性别套用“媳/婿/妇”后缀。

算法 5.2（配偶在末尾，骨架）.

```
mateLast(X + [M(g)]):
    if X = []:              return spouse(g)
    if X = [E/Y(s)]:        return siblingMate(E/Y, s, g)
    if all C:               return childMate(X, g)
    if X[0] = P:            return uncleMate(X, g)      # 伯母/姑丈/…
    if X[0] = E/Y:          return nephewMate(X, g)     # 侄媳/甥婿/…
    if X contains M:
        r = segmentReanchor(X, g)                       # 5.4: seg + rname
        if r: return r
    return suffixByLastSex(X, g)                        # 兜底：媳/婿/妇
```

例 5.1. $P_m E_m M_f$：父的兄的妻子。$X = P_m E_m$，$g = f$。X 的形状是 PE，直接查“伯叔姑舅姨的配偶”表，得“伯母”。

5.3 配偶在开头

设规范形为 $M(g)Y$。目标是配偶的亲属，因此

$$\mathrm{p}(M(g)Y) = \mathrm{p}(Y), \qquad \sigma(M(g)Y) = \sigma(Y).$$

命名时先把视角切到配偶，再按 Y 的形状命名；必要时在词首加“岳/公”等标记。

- $Y = P^t$：配偶的直系长辈。$t = 1$ 时是岳父/岳母、公公/婆婆；$t \geq 2$ 时用“岳/公”插入祖称，例如“曾祖岳父”“伯祖公父”。
- $Y = E/Y$：配偶的兄弟姐妹（大舅子/小姨子/大伯子/小姑子）。
- $Y = E/Y C^j$：配偶的侄甥（舅侄/姨甥、叔侄/姑甥）。
- $Y = P^k E/Y$：配偶的伯叔姑舅姨（伯岳父/姑岳母/舅岳父/姨岳母）。
- Y 含 M：递归切分，先段前缀、再重锚。

例 5.2. $M_f P_m P_m P_m$：妻子的父亲的父亲的父亲。$Y = P^3$，配偶女，按“岳”插入：曾祖岳父（+3、男）。$M_m P_m E_m$：丈夫的父亲的哥哥。$Y = PE$，配偶男，得“伯公父”（+2、男）。

5.4 配偶在中间：切分与重锚

设规范形为 $XM(g)Y$，且首尾都不是 M。取**第一个** M 作为切分点，令

$$s = \mathrm{p}(X).$$

目标是 X 的配偶的 Y，所以

$$\mathrm{p}(XM(g)Y) = s + \mathrm{p}(Y), \qquad \sigma(XM(g)Y) = \sigma(Y).$$

命名分两步：

$$\mathrm{seg} = \mathrm{segment}(X, g), \qquad \mathrm{rname} = \mathrm{reanchor}(g, s, Y).$$

段前缀函数不是一张会“查不到就放弃”的有限表，而是

$$\mathrm{segment}(X, g) = \begin{cases} \mathrm{table}(X, g), & \text{表命中,} \\ \mathrm{strip}(\mathrm{nameMateLast}(g, X)) + \text{眷/姻}, & \text{否则.} \end{cases}$$

其中 strip 去掉配偶名的后缀（媳妇、女婿、妻、夫、妇、媳、婿、嫂、母、父）。这样任意深度的 X 都至少有一个机械段前缀。切分与嵌套的示意见图 4.1。

算法 5.3（重锚名，按 Y 的形状）.

```
reanchor(g, s, Y, fallback=true):
    if Y = []:                      return ε
    if last(Y) = M(t):              # Y' 的配偶
        raw = reanchor(g, s, Y[0..-2], fallback)
        return addSpouseSuffix(raw, t)
    if Y contains M:
        i  = first index of M in Y
        Y1 = Y[0..i-1];  t = Y[i].g;  Y2 = Y[i+1..]
        seg2 = segment(Y1, t)       # 有限表未命中时用机械段前缀
        r2   = reanchor(t, s + p(Y1), Y2, true)
        if r2 = ε:
            r2 = termAtGeneration(s + p(Y), σ(Y))  # 保住内层路径
        return seg2 + r2
    if all P:                       return ancestorOrDescendantName(g, s, Y)
    if Y = [E/Y]:                   return siblingName(s, Y[0])
    if Y = P^k + E/Y:               return uncleName(s, Y)
    if Y = E/Y + C^j:               return nephewOrCousinName(s, Y)
    if Y = P + E/Y + C^j:           return cousinLineName(s, Y)
    if Y = P^k + E/Y + C^j:         # k >= 2
        gen = s + k - j
        k2  = gen + j
        if 1 <= k2 <= k:
            return cousinLine(pp[-k2:], E/Y, C^j)
        if k2 < 1 and gen <= -1:
            return termAtGeneration(gen, cg)
    if fallback:
        return termAtGeneration(s + p(Y), σ(Y))
    return ε                        # 顶层交给外层兜底
```

“嵌套”是这套算法最关键的一步。设 $Y = Y_1 M_2 Y_2$。内层配偶 M_2 的新锚点辈分不是 s，而是

$$s_2 = s + \mathrm{p}(Y_1),$$

因为从第一层配偶走到 Y_1 已经改变了辈分。内层命名结果为

$$\mathrm{seg}(Y_1, g_2) + \mathrm{rname}(g_2, s_2, Y_2).$$

如果错误地把 s_2 写成 s，就会出现上一版程序中的“曾外孙女眷舅曾祖眷姨祖父”一类错位：末位词比目标高了或低了一辈。

这里有三条必须同时成立的实现约束。第一，$\mathrm{seg}(Y_1, g_2)$ 在有限表未命中时必须走机械段前缀，不能返回空。第二，$\mathrm{rname}(g_2, s_2, Y_2)$ 在嵌套上下文中不能返回空；若形状未覆盖，至少返回目标的绝对辈分通用词。第三，外层拼接不能再把内层名字当成相对第一层配偶的偏移。三者缺一，都会出现“加一个词就突然变短”的系统性故障。

例 5.3（段前缀 + 重锚）. $P_mP_fE_fM_fE_mM_f$（父的母的姐的妻的兄的妻）：

- 外层：$X = P_mP_fE_f$（姨祖），$g = f$，$Y = E_mM_f$。
- $s = \mathrm{p}(X) = 2$。
- 段前缀：$X = PPE_f$，配偶女 → “姨祖眷”。
- 重锚：$Y = E_mM_f$ 以 M 结尾，先求 $Y' = E_m$ 的重锚名。$s = 2$，Y' 是单个 E_m，重锚名为“舅祖父”（+2、男）；再按配偶性别 f 把末字“父”换成“母”，得“舅祖母”。
- 主名：姨祖眷 + 舅祖母 = **姨祖眷舅祖母**（+2、女）。

例 5.3a（负辈分的内层重锚）. $Y_mC_fM_mE_fC_fC_mM_fP_mP_mE_fC_m$（弟的女的夫的姐的女的子的妻的父的父的姐的子）：

- 外层 $X = Y_mC_f$（侄女），$g = m$，$\mathrm{seg} =$ “侄姻”，$s = -1$。
- 内层 $Y_1 = E_fC_fC_m$（甥外孙），$\mathrm{seg}_2 =$ “甥外孙眷”，$s_2 = -3$。
- $Y_2 = P_mP_mE_fC_m = P^2EC$：$k = 2, j = 1$，$gen = -2$，$k_2 = gen + j = -1 < 1$。
- 目标男，$\mathrm{termAtGeneration}(-2, m) =$ “孙”。
- 主名：侄姻 + 甥外孙眷 + 孙 = **侄姻甥外孙眷孙**（-2、男）。

若这里返回空，外层只会剩下“侄姻”+“孙”，得到过短的“侄姻孙”。

例 5.3b（有限段前缀表的系统性缺口）. 比较两条只差末尾一个 E_f 的链：

- $P_mE_mM_fY_mC_fM_fP_fP_mP_fP_mE_mC_mM_f$ 以 M_f 结尾，走 mate-last，得“叔眷侄眷堂舅外曾祖母”。
- $P_mE_mM_fY_mC_fM_fP_fP_mP_fP_mE_mC_mM_fE_f$ 以 E_f 结尾，走 mate-middle。内层 $X' = P_fP_mP_fP_mE_mC_m = P^4EC$ 不在段前缀表中；旧实现返回空，外层退化为“叔眷曾祖母”。
- 修复后，机械段前缀为“堂舅外曾祖眷”，重锚为“姨曾祖母”，主名为 **叔眷侄眷堂舅外曾祖眷姨曾祖母**。

两条链只差一个词，却因为末尾是否为 M 而走不同分派；这正是“有限表 + 嵌套全有或全无”的系统性问题，而不是某一个词条的错误。

5.5 兜底与绝对辈分

段前缀与重锚名都曾被视为“部分函数”：表里没有就返回空。现在它们都有明确的兜底：段前缀用机械段前缀，嵌套重锚用目标绝对辈分的通用词。顶层仍保留“先再试一次重锚名，再取通用词”的兜底，以保持“妻曾祖父”这类既有结果不变。

算法 5.4（绝对辈分兜底）.

```
termAtGeneration(gen, sex):
    if gen = 0:    return 兄弟 / 姊妹
    if gen = -1:   return 男 / 女
    if gen <= -2:  return 孙名(-gen, sex)     # 孙/孙女，曾孙/曾孙女…
    else:          return 祖名(gen, sex)      # 父/母，祖父/祖母，曾祖父…

fallback(g, s, Y):
    tg = s + p(Y)
    return termAtGeneration(tg, σ(Y))
```

定理 5.2（兜底不变量）. 对任意 g, s, Y，`termAtGeneration(s+p(Y), σ(Y))` 的辈分等于 $s + \mathrm{p}(Y)$，性别等于 $\sigma(Y)$。

证明. 按 `termAtGeneration` 的分支逐一检查：0、-1、负辈分的孙名、正辈分的祖名，均由构造保证。□

例 5.4. $M_f P_f E_m M_m P_f P_m P_m E_m C_m$ 是一个段前缀表未覆盖的妻方深支。旧实现把它命名为“妻舅表外曾曾孙婿”（末位 -4，目标 +3）。新实现走兜底：$s = 0$，Y 的辈分为 +3，目标男，故末位取“曾祖父”，主名为“妻曾祖父”（+3、男）。这个词不如具体词漂亮，但辈分与性别正确。

5.6 正确性：辈分与性别不变量

定义 5.1（命名正确性）. 命名函数 $\mathrm{Name}(w) = T$ 是**正确**的，若

$$\mathrm{gen}(T) = \mathrm{p}(w), \qquad \mathrm{sex}(T) = \sigma(w).$$

若 $\mathrm{NF}(w)$ 有多条规范形，则每条规范形的命名都需正确。

定理 5.3（主定理）. 算法 5.1 – 5.4 对每个非零规范形都返回正确且有限的称呼；若走兜底，则兜底也正确。

证明梗概. 对规范形长度归纳。分派后，三个子算法分别处理：

- **直系：** `ancestors` / `descendants` 按剥离规则构造，辈分等于 P 的个数减去 C 的个数，性别取末位。
- **配偶在末尾：** 目标是 X 的配偶，辈分等于 $\mathrm{p}(X)$，性别为 g；各分支的配偶后缀保持辈分不变，性别由后缀决定。
- **配偶在开头/中间：** 目标的辈分为 $s + \mathrm{p}(Y)$，性别为 $\sigma(Y)$；段前缀只改变描述方式，不改变辈分；重锚名按第 4、5 类的公式构造，末位词满足目标辈分与性别；嵌套时递归使用 $s_2 = s + \mathrm{p}(Y_1)$，由归纳假设成立。
- **兜底：** 由定理 5.2。

终止性：重锚递归每次减少 Y 的长度；段前缀与词法表都是有限查表。□

唯一性 “正确”不等于“唯一”。同一个规范形在词法上可能有主名与别名（爸爸/父亲），也可能有多个合理分支（哥哥/自己/弟弟）。本书追求的是：主名稳定、分支可解释、每个分支都满足两条不变量。

5.7 关系转换：A 如何称呼 B

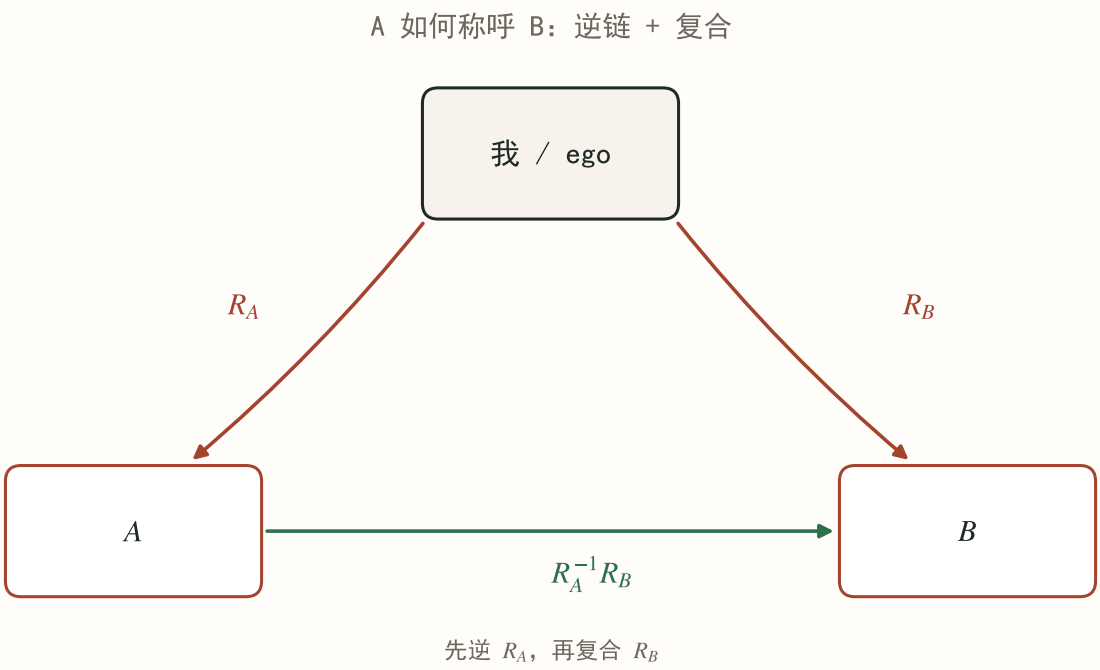


图 5.2 A 对 B 的称呼：先把“我→A”的链求逆，得到“A→我”，再复合“我→B”。

已知“我→A”的链 R_A 与“我→B”的链 R_B，要求 A 对 B 的称呼。做法是：

$$R_{A\to B} = R_A^{-1} R_B.$$

先求逆，再复合，再化简，最后命名。求逆要反序，并且每个生成元的类型与性别都要按“被描述者”重新标注。

定义 5.2（带类型逆）. 设链 $w = g_1 \cdots g_n$ 从 p_0 走到 p_n。逆链为

$$w^{-1} = g_n^{-1} \cdots g_1^{-1},$$

其中 g_i^{-1} 的类型是 g_i 的逆类型，性别取 g_i 的起点 p_{i-1} 的性别。具体地：

$$P_s^{-1} = C_{\mathrm{sex}(p_{i-1})}, \quad C_s^{-1} = P_{\mathrm{sex}(p_{i-1})}, \quad M_s^{-1} = M_{\mathrm{sex}(p_{i-1})},$$
$$E_s^{-1} = Y_{\mathrm{sex}(p_{i-1})}, \quad Y_s^{-1} = E_{\mathrm{sex}(p_{i-1})}, \quad I_s^{-1} = I_s.$$

对第一步，$p_0 = \mathrm{ego}$，其性别即“我”的性别。

例 5.5（爸爸称呼妈妈）. 设“我”为男。$R_A = P_m$（爸爸），$R_B = P_f$（妈妈）。求逆：

$$R_A^{-1} = C_m$$

（爸爸的孩子，男，即我）。复合：

$$R_A^{-1} R_B = C_m P_f.$$

化简 $C_m P_f$：子的母＝配偶或自己；P_f 的性别为女，得到 M_f 或 I_f。I_f 要求“我”为女，与“我”为男冲突，故只剩 M_f，即“妻子”。所以爸爸称呼妈妈为“妻子”。

例 5.6（堂哥称呼我）. 设“我”性别为 s。$R_A = P_m E_m C_m$（堂哥），$R_B = I$（我）。求逆要反序：

$$R_A^{-1} = P_m\, Y_m\, C_s$$

（堂哥的父←伯父，伯父的弟←父，父的子←我）。复合 $R_A^{-1} I = R_A^{-1}$，化简后得到“堂兄弟/堂姐妹”等分支；若“我”为男，则收窄为“堂兄弟”。

关系转换的完整实现还要处理“B 即 A 本人”的 I 分支：若 $B \neq A$，则含 I 的分支应被过滤；若 $B = A$，则直接返回“自己”。这部分在程序中由 `name_relate` 完成，本书不展开全部代码。

PART III · 第三部分

验证、实现与边界

第三部分把理论放回工程。三个实现、公共接口、独立检查器、一百一十万条穷举、二十万条长链随机测试，以及多组真实 BUG 的解剖。最后我们诚实地讨论这套系统的边界：哪些词只是机械构词，哪些关系超出公理，哪些结论来自机器证明，哪些只是经验验证。

第 6 章

实现、验证与边界

Implementation, Verification, and Limits

理论必须能落地。本章介绍三个实现（Python 引擎、Python 命名器、JavaScript 演示）、公共接口与测试方法，然后解剖几组真实 BUG，并讨论从“点状补丁”到“系统性兜底”的演化。最后讨论这套系统的边界：哪些词是机械构词，哪些关系超出公理，哪些结论只是经验验证而非机器证明。

6.1 三个实现与公共接口

表 6.1 实现分层

文件	职责	关键接口
`generators.py`	生成元类	`Parent`, `Child`, `Mate`, `Elder`, `Younger`
`kinship_formula.py`	关系代数、化简	`init`, `priority`, `kinship_type`
`naming.py`	词法表与命名	`naming`, `naming_full`, `name_chain`, `invert_gens`, `name_relate`
`demo/kinship.js`	JavaScript 移植	`init`, `nameChain`, `relate`
`demo/index.html`	网页计算器	快速计算、变量、关系转换

Python 侧的命名入口是 `name_chain(gens)`：输入生成元列表，输出

$$\{\text{chain, formula, names, primary, notes}\}.$$

`names` 的每一项含 `monomial`、`primary`、`alternatives` 三个字段。若化简得到多条规范形，`primary` 用“ / ”连接；每条规范形的主名分别可查。JavaScript 侧的 `nameChain(gens, egoG)` 返回同构的对象，字段名略有差异。

```
from generators import Parent, Child, Mate, Elder, Younger
from naming import name_chain

gens = [Parent('m'), Parent('f'), Elder('f'), Mate('f'), Elder('m'), Mate('f')]
r = name_chain(gens)
print(r['formula'])   # PmPfEfMfEmMf
print(r['primary'])   # 姨祖眷舅祖母
```

6.2 穷举与随机验证

我们用一个独立的检查器验证命名结果，而不是只和 `relationship` 库对照。检查器的做法是：

1. 对每条规范形，取生成名的最后一个“眷/姻”之后的末位词；
2. 标准词用 `relationship` 库解析，项目自造的“眷/姻/叔男/姑女/孙男”等用自定义规则补全；
3. 把末位词的辈分、性别与 `priority(公式)`、`formula.gender` 比较；
4. 不一致或无法解析的记录为失败。

表 6.2 验证结果

测试	规模	结果
穷举全部 1 – 6 段链	1,111,110 条	0 处辈分/性别错位
随机 6 – 10 段长链（第一组）	150,000 条	0 处
随机 6 – 10 段长链（第二组）	50,000 条	0 处
上一轮失败的 6 段链复测	896 条	0 条剩余
Python/JS 长链对照	305 条	逐字一致

为什么不用 relationship 库做唯一标准 该库不支持同性婚姻，也不覆盖深层“眷/姻”组合；它的词表本身是人工整理的，存在缺口。我们只用它解析标准词，不用它判断项目自造词。真正的判据是第 5 章的两条不变量：末位辈分与末位性别。

6.3 典型 BUG 与系统性兜底

6.3.1 案例 A：嵌套配偶把相对辈分当成绝对辈分

输入. $C_mC_fC_fM_fP_mP_fP_fE_mM_fY_f$（子的女的女的妻的父的母的母的兄的妻的妹）。目标辈分 0，性别女。
旧输出. 曾外孙女眷舅曾祖眷姨祖父。新输出. 曾外孙女眷舅曾祖眷妹。

旧实现把 Y 中的第二个配偶当成“第一层配偶的亲戚”，用 `name_km(Y)` 整体命名，得到末位“姨祖父”（+2、男）。但目标是 0 辈、女。修复分两步：一是给重锚函数增加“Y 中还有配偶”的递归分支，二是把单 E/Y 的 $gen \geq 3$ 分支从“$gen-1$ 代 + 固定父字”改成“gen 代 + 目标性别”。修复后末位是“妹”（0、女），与目标一致。

6.3.2 案例 B：同性段前缀缺口与“哥哥媳妇”

输入. $P_mP_fE_fM_fE_mM_f$（父的母的姐的妻的兄的妻）。目标 +2，女。
旧输出. 姨奶奶眷哥哥媳妇。新输出. 姨祖眷舅祖母。

旧实现的“段前缀 + 重锚”分支只对 X 以 C 开头生效；这里 $X = P_mP_fE_f$，被跳过，落到通用后缀兜底：先把 X 命名为“姨奶奶眷哥哥”，再因为末位是男而机械加“媳妇”。更深一层的原因是段前缀表没有同性组合：PPE_f 配女配偶时返回空，本应给出“姨祖眷”。修复补全同性段前缀表，把“段前缀 + 重锚”分支泛化到任意含 M 的 X，并给口语兄弟姐妹词加“嫂子/弟媳”替换。于是末位是“舅祖母”（+2、女）。

6.3.3 案例 C：负辈分被写成“父/伯父”

输入. $C_mC_mC_mM_mP_mE_m$（曾孙的丈夫的父亲的哥哥）。目标 -2，男。
旧输出. 曾孙姻伯父。新输出. 曾孙姻孙。

$Y = P_mE_m$ 是 P^kE/Y 形状，$s=-3, k=1$，所以 $gen = s+k = -2$。旧实现只处理 $gen = 0, -1, 1, 2, \geq 3$，负辈分落到兜底，用 Y 的相对名字“伯父”（+1、男）。修复补上 $gen \leq -2$ 的绝对孙名分支：-2 代男取“孙”。同类修复还覆盖 $Y = P^t$ 与 $Y = E/YC^j$ 的负辈分。

6.3.4 案例 D：配偶开头深支被误交给表亲配偶函数

输入. $M_fP_fE_mM_mP_fP_mP_mE_mC_m$（妻方深支）。目标 +3，男。
旧输出. 妻舅表外曾曾孙婿（末位 -4）。新输出. 妻曾祖父（+3、男）。

这条链以 M_f 开头，进入 `name_mate_first`。尾部含一个以 M_m 开头的子链，旧实现把 `tail[:-1]` 交给 `name_cousin_mate`，而该函数期望“纯 C 链 + 末位 M”，于是产出一个辈分完全错位的词。修复加了两道 guard：若重锚失败且 `tail` 不是“纯 C + 末位 M”，就直接按绝对辈分兜底；“妻曾祖父”虽然不如具体词漂亮，但辈分、性别都正确。

6.3.5 案例 E–F：点状补丁之后，为什么还会漏

前四个案例各自补了一个形状。但后续又出现两条链，说明“逐个补形状”本身不是可靠的架构：只要段前缀表仍是有限表，嵌套重锚又要求“段前缀和重锚名都非空”，换一个深度就会从新的缺口漏出。

E. 负辈分的 $k_2 < 1$

> 输入. $Y_m C_f M_m E_f C_f C_m M_f P_m P_m E_f C_m$。目标 -2，男。
> 旧输出. 侄姻孙。新输出. 侄姻甥外孙眷孙。

外层 $X = Y_m C_f$（侄女），$s = -1$，段前缀“侄姻”；内层 $Y_1 = E_f C_f C_m$（甥外孙），$s_2 = -3$，段前缀“甥外孙眷”；$Y_2 = P^2EC$，$k = 2, j = 1$，$gen = -2$，$k_2 = gen + j = -1 < 1$。旧版只处理 $1 \le k_2 \le k$，于是 Y_2 返回空，嵌套失败，外层只剩“侄姻”+“孙”。修复新增 $k_2 < 1$ 且 $gen \le -1$ 的通用后代词分支；内层段因此保留。

F. 有限段前缀表 + 嵌套“全有或全无”

> 输入 A. $P_m E_m M_f Y_m C_f M_f P_f P_m P_f P_m E_m C_m M_f$。目标 +3，女。
> 输出 A. 叔眷侄眷堂舅外曾祖母。
> 输入 B. 在 A 末尾加 E_f：$P_m E_m M_f Y_m C_f M_f P_f P_m P_f P_m E_m C_m M_f E_f$。
> 旧输出 B. 叔眷曾祖母。新输出 B. 叔眷侄眷堂舅外曾祖眷姨曾祖母。

A 以 M_f 结尾，走 mate-last，先给前面的 X 整体命名，再加配偶后缀，所以长路径保留。B 以 E_f 结尾，走 mate-middle；内层 $X' = P_f P_m P_f P_m E_m C_m = P^4EC$ 不在有限段前缀表中，旧实现返回空，嵌套分支整体放弃，外层退化为“叔眷”+通用词“曾祖母”。修复后，段前缀表未命中时用机械段前缀：

$$\mathrm{strip}(\mathrm{nameMateLast}(f, X')) + \text{眷} = \text{堂舅外曾祖} + \text{眷},$$

嵌套重锚再给出“姨曾祖母”，主名变为长路径。

这两个案例的共同点不是某个词条写错，而是**函数的全域性**：段前缀必须是全函数（或至少对非空 X 有机械兜底），嵌套重锚不能返回空，否则外层就没有机会保留内层路径。第 5.4 节的三条实现约束就是为此而写的。

6.4 边界与已知限制

- 机械构词. “眷/姻”体系中的“男眷叔兄”“孙姻孙”“叔男妇”等不是日常口语，而是为了给任意长链一个唯一字符串而设计的段标记。它们满足辈分/性别不变量，但不保证在某个方言里真实使用。
- 兜底的泛化. 极深且段前缀表未覆盖的形状会用机械段前缀 + 绝对辈分通用词，得到“叔眷侄眷堂舅外曾祖眷姨曾祖母”这类长词。它保留路径、保证辈分/性别正确，但不保证具体、自然；若要更具体，需要继续扩充词表。
- 公理的理想化. 十三条公理假设每人恰有两个父母、配偶唯一、人人有子女与配偶；不处理收养、继亲、离婚再婚、多配偶等情形。同性婚姻是通过“性别本位”外推的，不是对现实婚姻制度的完整建模。
- 年龄信息. E/Y 只记录兄弟姊妹之间的长幼；堂表亲的长幼、侄甥的长幼都不在链中。因此主名用“堂兄弟”“侄”等合并词，别名才给出“堂哥/堂弟”。
- 形式验证的范围. 无性别版本的十三条公理与主要定理已在 Lean 4 中机器验证；带类型的命名算法没有完整的机器证明，其正确性由不变量证明梗概与穷举/随机测试支持。
- 唯一性. 同一规范形可能有主名与别名，也可能有多个合理分支；本书保证的是“每条分支正确”，不是“全局唯一字符串”。

6.5 练习与核对清单

> 练习 1. 把“爸爸的姐姐的丈夫的弟弟”写成链，并命名。
> 答案. $P_m E_f M_m Y_m$。化简：$P_m E_f$ 是大姑，M_m 是姑丈，Y_m 是姑丈的弟弟；命名走“配偶在中间”，段前缀“姑姻”，重锚名“叔父”，结果为 姑姻叔父（+1、男）。

练习 2. 化简 $P_mE_mC_mP_m$，并解释为什么有两个分支。

答案. $P_mE_mC_mP_m \to P_mE_mM_m$ 或 P_mE_m，即“伯婿 / 伯父”。前者是伯父的丈夫（同性婚姻分支），后者是伯父本人；两条都保留，因为代数不区分“生父”与“配偶”。

练习 3. 证明 $CP = M + I$。

提示. 见定理 2.12。关键是用 A5 把“共同子女”变成“配偶”，用 A12 保证配偶有子女。

练习 4. 为什么 $E_mM_mM_f = 0$？

答案. E_mM_m 是哥哥的丈夫（男）；再 M_f 要求这位丈夫的妻子是女，但丈夫的配偶应是哥哥或“我”（男），性别冲突，故整支为 0。

练习 5. 给 $C_mC_mC_mM_mP_mE_m$ 命名，并说明末位词的辈分与性别。

答案. 曾孙姻孙；末位“孙”是 −2 辈、男，与目标一致。

核对清单 （1）先化简，再命名；（2）看末位是不是 M，再决定分派；（3）含 M 的链先切第一个 M；（4）嵌套配偶的新锚点辈分是 $s + \mathrm{p}(Y_1)$，不是 s；（5）末位词的辈分必须等于 $\mathrm{p}(w)$，性别必须等于 $\sigma(w)$；（6）若走兜底，用 `termAtGeneration`，不要拼相对名字。

附录 A

化简表与不可约对

本附录把第 2、3 章的改写规则集中列出，便于手算与实现对照。左列是“原式”，右列是“可改写为”，备注给出依据或性别条件。

表 A.1　无性别化简表

原式	改写为	依据	备注
PC	$S+I$	定理 2.8	父母的子女
SS	$S+I$	定理 2.9	同胞的同胞
PM	P	定理 2.10	父母的配偶
SP	P	定理 2.10	同胞的父母
CS	C	定理 2.11	子女的同胞
CP	$M+I$	定理 2.12	子女的父母
MC	C	定理 2.13	配偶的子女
MM	I	定理 2.13	配偶的配偶
EY	$S+I$	定理 2.14	兄姐的弟妹
YE	$S+I$	定理 2.14	弟妹的兄姐

表 A.2　带类型改写表（$s,t\in\{m,f\}$）

A	B	改写为	条件/说明
P_s	C_t	$E_t,\ I_t,\ Y_t$	三条分支
P_s	M_t	P_t	配偶并到父/母
M_s	C_t	C_t	配偶的子女
M_s	M_t	I_t	t 必须等于前一人的性别，否则 0
C_s	P_t	$M_t,\ I_t$	两条分支
C_s	E_t	C_t	子的兄
C_s	Y_t	C_t	子的弟
E_s/Y_s	P_t	P_t	兄弟姊妹的父母
E_s	E_t	E_t	兄的兄
Y_s	Y_t	Y_t	弟的弟
E_s	Y_t	$E_t,\ I_t,\ Y_t$	三条分支
Y_s	E_t	$E_t,\ I_t,\ Y_t$	三条分支

幺元与零元：$I_sX=X$；$XI_s=X$ 当 $\sigma(X)=s$，否则 0；$0X=X0=0$。

附录 A 定理. 不可再化简的二元组恰有八个：

$$PP,\quad PS,\quad MP,\quad MS,\quad SM,\quad CM,\quad SC,\quad CC.$$

表 A.3　八个不可约对与日常关系

二元组	日常关系	例
PP	祖父母	P_mP_m 爷爷
PS	父母的兄弟姐妹	P_mE_m 伯父
MP	配偶的父母	M_fP_m 岳父
MS	配偶的兄弟姐妹	M_fE_m 大舅子

二元组	日常关系	例
SM	兄弟姐妹的配偶	E_mM_f 嫂子
CM	子女的配偶	C_mM_f 儿媳
SC	兄弟姐妹的子女	E_mC_m 侄子
CC	孙辈	C_mC_m 孙子

附录 B

眷姻段前缀表

段前缀的通用构造是：基词由 X 的性别决定，后缀由配偶性别决定。配偶女用“眷”，配偶男用“姻”。下表列出常用基词；完整表见《亲属称呼手册》附录 C。

表 B.1　常用段前缀基词

X 的形状	基词	例（配偶女/配偶男）
C_m	男	男眷 / 男姻
C_f	女	女眷 / 女姻
E/Y	兄弟 / 姊妹	兄弟眷 / 姊妹姻
C_mC_m	孙	孙眷 / 孙姻
C_mC_f	孙	孙眷 / 孙姻
C_fC_m	外孙	外孙眷 / 外孙姻
C_fC_f	外孙	外孙眷 / 外孙姻
E/Y_mC_m	侄	侄眷 / 侄姻
E/Y_fC_m	甥	甥眷 / 甥姻
P_mE/Y_m	叔	叔眷 / 叔姻
P_fE/Y_m	舅	舅眷 / 舅姻
P_mE/Y_f	姑	姑眷 / 姑姻
P_fE/Y_f	姨	姨眷 / 姨姻
M_mE/Y_m	叔兄弟	叔兄弟眷 / 叔兄弟姻
M_fE/Y_m	舅兄弟	舅兄弟眷 / 舅兄弟姻
M_mE/Y_f	姑姊妹	姑姊妹眷 / 姑姊妹姻
M_fE/Y_f	姨姊妹	姨姊妹眷 / 姨姊妹姻

高辈段 P^2E/Y 的基词为“叔祖/姑祖/舅祖/姨祖/叔外祖/姑外祖/舅外祖/姨外祖”，再加眷/姻。例如 $P_mP_fE_f$ 配女配偶 → 姨祖眷；配男配偶 → 姨祖姻。更深的 P^kE/Y 用“标记 + 外 + 祖辈级”构造。

附录 C

重锚名速查

本附录是第 5.4 节 `reanchor(g,s,Y)` 的速查表。所有公式都以保证末位辈分/性别为目标；若某形状不在表中，嵌套调用用机械段前缀/绝对辈分兜底，不再直接返回空。

表 C.1　重锚名按 Y 的形状分类

Y 的形状	辈分	词形/规则

Y 的形状	辈分	词形/规则
以 $M(t)$ 结尾：$Y'M(t)$	同 Y'	先求 Y' 的重锚名，再按 t 加“妇/婿”或改末字“父/母”
全是 P：P^t	$gen = s + t$	$s \geq 1$：祖名(g+m^{s−1}+性别序列)；$s = 0$：祖名(性别序列)；$s < 0$：按 gen 取孙名
单个 E/Y	$gen = s$	0：兄/弟/姊/妹；1：舅/姨/伯/叔/姑；2：舅祖父/姨祖母/…；-1：男/女；-2：孙男/孙女；≤-3：曾…孙；≥3：舅/姨/叔/姑 + 祖名
P^kE/Y	$gen = s + k$	0：flav + 兄弟/姊妹；-1：flav + 男/女；1：伯/叔/姑/舅/姨；2：flav + 祖父/祖母；≤-2：孙名(gen)；≥3：flav + 祖名(seq)
E/YC^j	$gen = s - j$	0：line + 兄/姊；1：舅表/姨表/堂伯/堂姑；$s = 0$：侄/甥 + 男/女；-1：男/女；≤-2：孙名(gen)；≥2：line + 伯叔/姑 + 祖名
PE/YC^j	$gen = s + 1 - j$	0：line + 兄/姊；-1：flav + 男/女；≤-2：孙名(gen)；1：堂舅/堂姨/堂伯/堂姑；≥2：line + 伯叔/姑 + 祖名
P^kE/YC^j（$k \geq 2$）	$gen = s + k - j$	$k_2 = gen + j$；$1 \leq k_2 \leq k$：取最近 k_2 个 P，按第 6 章堂表支命名；$k_2 < 1$ 且 $gen \leq -1$：$termAtGeneration(gen, cg)$；$k_2 > k$：空，交给外层
嵌套：$Y_1M_2Y_2$	$s_2 = s + \mathrm{p}(Y_1)$	$\mathrm{seg}(Y_1, g_2) + \mathrm{rname}(g_2, s_2, Y_2)$；seg 未命中用机械段前缀，嵌套 rname 不得为空

段前缀表未命中时，不能直接返回空。算法用 $\mathrm{strip}(\mathrm{nameMateLast}(g, X)) + $眷/姻 作为机械段前缀； 嵌套重锚若仍无词形，则用 $termAtGeneration$ 给出目标绝对辈分的通用词。 这样深度超出有限表的 X 也不会丢失整段路径。

附录 D

术语与符号

表 D.1 术语表

术语	含义
生成元	P, C, M, E, Y（带性别时为 P_s 等）
链 / 词	生成元的有限序列 $g_1 \cdots g_n$
规范形	栈式化简算法不能再改写的输出
辈分 p	$\#P - \#C$；正长辈、负晚辈、零平辈
目标性别 σ	链末位生成元的性别
段前缀 seg	描述 X 与配偶这一段的标记
重锚名 rname	把视角切到配偶后给 Y 的名字
眷 / 姻	中间配偶为女 / 男
主名 / 别名	计算器结果的第一项 / 备选词
多解	一条链化简出多条规范形
绝对辈分兜底	按 $s + \mathrm{p}(Y)$ 直接取通用词

附录 E

Lean 4 机器验证

《亲属关系公理》一文的全部定理已在 Lean 4 中机器验证，不依赖 Mathlib，报告为 0 错误、0 `sorry`、0 `axiom`。验证覆盖的是无性别的关系代数与十三条公理：每人恰有两个父母、全同胞、长幼严格全序，以及 $PC, SS, PM, SP, CS, CP, MC, MM, EY, YE$ 十条等式。

表 E.1 定理对照（节选）

本文	Lean 名称
定理 2.2（恰有两个父母）	`Kin13.exactly_two_parents`
定理 2.4（全同胞）	`Kin13.sib_full`
定理 2.5 – 2.7（长幼全序）	`Kin13.not_elder_self`, `elder_asymm`, `elder_total`
定理 2.8 – 2.14（化简等式）	`Kin13.PC_eq`, `SS_eq`, `PM_eq`, `SP_eq`, `CS_eq`, `CP_eq`, `MC_eq`, `MM_eq`, `EY_eq`, `YE_eq`
定理 2.17（人口无限）	`COMBINED.md` §2

验证边界. Lean 验证的是无性别公理系统；本书第 4、5 章的带类型词法与命名算法没有完整的机器证明。带类型系统的正确性由三部分支持：第 3 章的终止性证明、第 5 章的不变量证明梗概，以及第 6 章的穷举/随机测试。读者若要把命名算法也形式化，需要把词法表与“段前缀/重锚”写成归纳定义，并证明每个分支的末位辈分/性别；这是一项有意义但独立的工作。

> 复现 无性别验证的复现命令见《亲属关系公理》附录 A.3；命名层的复现命令见项目根目录的 Python/JS 测试脚本。本书的图表由 `book/figures.py` 生成，PDF 由 HTML + KaTeX + 无头浏览器打印生成。

参考与来源

- Subaru Akuru. *PCMS-Kin：亲属关系代数与中文称呼引擎*. Python/JavaScript 实现，MIT License，2026.
- 《亲属关系公理》. 十三条公理与 Lean 4 机器验证，2026.
- mumuy/relationship（Python 移植版）. 中文亲属称谓词表与选择器；本书仅用它解析标准词，不作为深层/同性关系的唯一标准.
- KaTeX 0.16. 数学排版；Matplotlib. 矢量插图；PyMuPDF. PDF 元数据与书签；Microsoft Edge. HTML 打印.